

IMPLEMENTASI METODE SCRUM DALAM PERANCANGAN SISTEM INFORMASI SARANA DAN PRASARANA KAMPUS BERBASIS WEB MENGGUNAKAN CRUD RDBMS

LAPORAN PROYEK II

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2 pada Program Studi DIV
Teknik Informatika



DISUSUN OLEH :

714240008 MUHAMMAD ARIF RIVALDI
714240041 RADITYA RIZKI RAHARJA

PROGRAM STUDI DIV TEKNIK INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS LOGISTIK DAN BISNIS INTERNASIONAL
BANDUNG

2026

LEMBAR PENGESAHAN

IMPLEMENTASI SISTEM INFORMASI PEMINJAMAN SARANA DAN PRASARANA KAMPUS BERBASIS WEB MENGGUNAKAN CRUD RDBMS

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2
Program Studi D-IV Teknik Informatika

Disusun Oleh:

Muhammad Arif Rivaldi 714240008
Raditya Rizki Raharja 714240041

Dosen Pembimbing

Koordinator Proyek 2

Rolly Maulana Awangga, S.T., MT., CAIP, SFPC
NIK. 117.86.219

Rd. Nuraini Siti Fathonah, S.S., M.Hum., SFPC
NIK. 118.72.251

Menyetujui
Ketua Program Studi DIV Teknik Informatika

Roni Andarsyah, S.T., M.Kom., SFPC
NIK. 115.88.193

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Muhammad Arif Rivaldi
NPM : 714240008
Program Studi : D IV Teknik Informatika
Judul : Implementasi Sistem Informasi Peminjaman Sarana dan Prasarana
Kampus Berbasis Web Menggunakan CRUD RDBMS

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik & Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Muhammad Arif Rivaldi

NIM. 714240008

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Raditya Rizki Raharja
NPM : 714240041
Program Studi : D IV Teknik Informatika
Judul : Implementasi Sistem Informasi Peminjaman Sarana dan Prasarana
Kampus Berbasis Web Menggunakan CRUD RDBMS

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik & Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Raditya Rizki Raharja

NIM. 714240041

ABSTRAK

Pengelolaan sarana dan prasarana kampus secara manual menimbulkan permasalahan seperti kesulitan pencatatan data dan bentrok jadwal peminjaman. Penelitian ini bertujuan merancang Sistem Informasi Peminjaman Sarana dan Prasarana Kampus berbasis web menggunakan *CRUD* RDBMS. Sistem dikembangkan dengan metode *Scrum*, dibangun menggunakan Go untuk *backend* dan PostgreSQL sebagai basis data. Fitur utama meliputi pengajuan peminjaman, verifikasi oleh Sarpras, validasi kehadiran oleh Security, serta kalender ketersediaan ruangan. Pengujian *Black Box Testing* menunjukkan seluruh fungsi berjalan sesuai spesifikasi dengan *code coverage* modul inti di atas 75%. Sistem yang dikembangkan mampu mengintegrasikan proses peminjaman secara efektif dan terdokumentasi dengan baik.

Kata Kunci: Sistem Informasi, Peminjaman Sarana Prasarana, *CRUD*, RDBMS, *Scrum*

ABSTRACT

Manual management of campus facilities causes problems such as data recording difficulties and scheduling conflicts. This research aims to design a web based Campus Facilities Loan Information System using CRUD RDBMS. The system was developed using the Scrum method, built with Go for backend and PostgreSQL as database. Main features include loan applications, verification by Sarpras, attendance validation by Security, and room availability calendar. Black Box Testing showed all functions ran according to specifications with core module code coverage above 75%. The developed system successfully integrates the facilities loan process effectively and well documented.

Keywords: *Information System, Facilities Loan, CRUD, RDBMS, Scrum*

KATA PENGANTAR

Puji dan syukur ke hadirat Allah SWT yang telah melimpahkan rahmat, hidayah, dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Proyek II dengan judul **“Implementasi Metode Scrum Dalam Perancangan Sistem Informasi Sarana Dan Prasarana Kampus Berbasis Web Menggunakan CRUD RDBMS”** sebagai salah satu syarat kelulusan mata kuliah Proyek II pada Program Studi DIV Teknik Informatika, Sekolah Vokasi, Universitas Logistik dan Bisnis Internasional.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, penulis menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Rolly Maulana Awangga, S.T., M.T., CAIP, SFPC. selaku dosen pembimbing Proyek II yang telah memberikan arahan, bimbingan, dan masukan selama proses pengerjaan proyek ini.
2. Staf Sarana dan Prasarana Universitas Logistik dan Bisnis Internasional yang telah bersedia memberikan waktu dan informasi dalam proses wawancara dan pengumpulan kebutuhan sistem.
3. Orang tua dan keluarga yang senantiasa memberikan dukungan moril maupun materiil.
4. Rekan-rekan mahasiswa Program Studi Teknik Informatika yang telah memberikan dukungan dan kerja sama selama pengerjaan proyek ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan pihak-pihak yang berkepentingan.

DAFTAR ISI

LEMBAR PENGESAHAN	i
SURAT PERNYATAAN	ii
ABSTRAK	iv
<i>ABSTRACT</i>	v
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR GAMBAR	x
DAFTAR TABEL	xi
BAB 1 PENDAHULUAN	1
1.1 Deskripsi Aplikasi	1
1.2 Rumusan Masalah	2
1.3 Tujuan	3
1.4 Lingkup Dokumentasi	4
BAB 2 LANDASAN TEORI	5
2.1 Tinjauan Pustaka	5
2.2 Sistem Informasi dan Manajemen Sarana Prasarana	6
2.3 Metodologi <i>Scrum</i>	7
2.4 Basis Data Relasional dan Konsep <i>CRUD</i>	8
2.5 Teknologi Sistem Informasi Berbasis Web dengan Prinsip <i>Green Computing</i>	8
2.6 Pengujian Perangkat Lunak	9
BAB 3 METODOLOGI PENELITIAN	11
3.1 Metode Penelitian	11
3.2 Kerangka Penelitian Berdasarkan <i>PICOC</i>	12
3.3 Tahapan Pengembangan Sistem Menggunakan Metode <i>Scrum</i>	13
3.3.1 Penetapan <i>Sprint Goal</i> dan <i>Product Backlog</i>	14
3.3.2 Perencanaan <i>Sprint (Sprint Planning)</i>	14

3.3.3	Pelaksanaan <i>Sprint</i> (<i>Sprint Execution</i>)	15
3.3.4	Evaluasi <i>Sprint</i> (<i>Sprint Review</i>)	16
3.3.5	Evaluasi Proses Pengembangan (<i>Sprint Retrospective</i>)	16
3.4	Analisis Kebutuhan Sistem	16
3.4.1	Identifikasi Aktor Sistem	17
3.4.2	Kebutuhan Fungsional Sistem	17
3.4.3	Kebutuhan Non-Fungsional Sistem	18
3.4.4	Aturan Bisnis (<i>Business Rules</i>)	18
3.5	Perancangan Sistem	19
3.5.1	<i>Usecase Diagram</i>	19
3.5.2	<i>Flowchart Diagram</i>	21
3.5.3	<i>Sequence Diagram</i>	25
3.5.4	<i>Entity Relationship Diagram</i> (ERD)	29
3.5.5	<i>Class Diagram</i>	30
3.5.6	<i>Business Process Model and Notation</i> (BPMN) <i>Diagram</i>	32
BAB 4	IMPLEMENTASI DAN PENGUJIAN	36
4.1	Lingkungan Implementasi Sistem	36
4.1.1	Perangkat Keras	36
4.1.2	Perangkat Lunak	36
4.2	Implementasi Desain Tampilan	37
4.2.1	Halaman Utama (<i>Guest</i>)	37
4.2.2	Halaman <i>Login</i>	38
4.2.3	Halaman <i>Register</i>	39
4.2.4	<i>Dashboard</i> Mahasiswa	40
4.2.5	<i>Form</i> Pengajuan Peminjaman	41
4.2.6	<i>Dashboard</i> Sarpras	42
4.2.7	<i>Form</i> Verifikasi Peminjaman	43
4.2.8	Halaman Laporan Peminjaman	43
4.2.9	<i>Dashboard Security</i>	44
4.2.10	<i>Form</i> Verifikasi Kehadiran	46
4.3	Pengujian	46
4.3.1	Pengujian <i>Black Box</i>	46
4.3.2	Pengujian <i>Code Coverage</i>	47
4.4	<i>Repository GitHub Project</i>	48
BAB 5	KESIMPULAN DAN SARAN	49
5.1	Kesimpulan	49

5.2	Saran	50
	DAFTAR PUSTAKA	51
	LAMPIRAN	54
	GLOSARIUM	57

DAFTAR GAMBAR

3.1	<i>Alur Penerapan Metode Scrum</i>	13
3.2	<i>Usecase Diagram</i> Sistem	20
3.3	<i>Flowchart Diagram</i> Alur Pengajuan Peminjaman	21
3.4	<i>Flowchart Diagram</i> Alur Verifikasi Peminjaman	23
3.5	<i>Flowchart Diagram</i> Alur Verifikasi Kehadiran Peminjam	24
3.6	<i>Sequence Diagram</i> Pengajuan Peminjaman	26
3.7	<i>Sequence Diagram</i> Verifikasi Peminjaman	27
3.8	<i>Sequence Diagram</i> Verifikasi Kehadiran	28
3.9	<i>Entity Relationship Diagram (ERD)</i>	29
3.10	<i>Class Diagram</i> Sistem	31
3.11	<i>BPMN Diagram</i> Proses Peminjaman	32
3.12	<i>BPMN Diagram</i> Verifikasi Peminjaman	34
3.13	<i>BPMN Diagram</i> Verifikasi Kehadiran	35
4.1	Tampilan Halaman Utama	38
4.2	Tampilan Halaman <i>Login</i>	39
4.3	Tampilan Halaman <i>Register</i>	39
4.4	Tampilan <i>Dashboard</i> Mahasiswa	40
4.5	Tampilan <i>Form</i> Pengajuan	41
4.6	Tampilan <i>Dashboard</i> Sarpras	42
4.7	Tampilan <i>Form</i> Verifikasi Peminjaman	43
4.8	Tampilan Halaman Laporan Peminjaman	44
4.9	Tampilan <i>Dashboard Security</i>	45
4.10	Tampilan <i>Form</i> Verifikasi Kehadiran	46
L.1	Ringkasan <i>Code Coverage Backend</i>	54
L.2	Detail <i>Coverage Auth Service</i>	55
L.3	Detail <i>Coverage Kehadiran Service</i>	55
L.4	Detail <i>Coverage Peminjaman Service</i>	56
L.5	Dokumentasi Wawancara dengan Staf Sarana dan Prasarana	56

DAFTAR TABEL

3.1	Kerangka <i>PICOC</i> Penelitian	12
3.2	Rincian Pelaksanaan <i>Sprint</i>	15
3.3	Relasi Antar Entitas	30
3.4	Rincian Kelas dalam Sistem	31
4.1	Pengujian Aplikasi	47
4.2	Ringkasan Hasil <i>Code Coverage Backend</i>	48

BAB 1

PENDAHULUAN

1.1 Deskripsi Aplikasi

Perkembangan teknologi informasi telah mendorong institusi pendidikan tinggi untuk melakukan transformasi digital dalam pengelolaan sumber daya kampus, termasuk sarana dan prasarana (Ramdany and Bahari 2024). Pada praktiknya, proses peminjaman sarana dan prasarana di banyak perguruan tinggi masih dilakukan secara manual atau semi-digital, seperti pencatatan berbasis formulir, *spreadsheet* terpisah, atau komunikasi tidak terintegrasi (Nabella et al. 2025). Kondisi ini menimbulkan berbagai permasalahan, antara lain keterlambatan proses, redundansi data, dan potensi bentrok jadwal peminjaman (Purwati, Pratama, and Rapiyanta 2022), serta rendahnya akurasi informasi yang menyulitkan pelaporan dan evaluasi secara *real-time* (Velasquez-Jimenez et al. 2025).

Sejalan dengan *state of the art* pengembangan sistem informasi basis web, pendekatan iteratif dan adaptif menjadi kebutuhan utama agar sistem dapat berkembang sesuai kebutuhan pengguna yang dinamis. Metode *Scrum*, sebagai salah satu kerangka kerja *Agile*, banyak digunakan dalam pengembangan perangkat lunak modern karena kemampuannya dalam meningkatkan kolaborasi tim (Nugraha and Hardian 2021). Selain itu, metode ini dipilih karena menawarkan fleksibilitas terhadap perubahan kebutuhan (Rifky, Mursityo, and Prakoso 2022), serta memfasilitasi percepatan siklus pengembangan (Permana 2015). Namun, penerapan *Scrum* pada pengembangan sistem informasi peminjaman sarana dan prasarana kampus masih memiliki tantangan tersendiri, terutama terkait adaptasi proses di dalam organisasi (Putrianasari et al. 2024).

Di sisi lain, sistem informasi peminjaman yang efisien membutuhkan perancangan basis data yang baik untuk menjamin konsistensi dan keandalan data. Penerapan teknik normalisasi basis data menjadi aspek penting dalam meminimalkan redundansi serta meningkatkan efisiensi penyimpanan dan pemrosesan data (Purwati, Pratama, and Rapiyanta 2022). Selain itu, kualitas sistem yang dikembangkan perlu dievaluasi secara sistematis, misalnya melalui metode *black box testing* untuk memastikan fungsionalitas sistem sesuai kebutuhan pengguna (Luh et al. 2020), serta pengukuran kinerja untuk menilai keandalan teknis sistem (Pratama and Farisi 2025).

Sebagai solusi atas permasalahan tersebut, penelitian ini mengusulkan pengembangan sistem informasi peminjaman sarana dan prasarana kampus berbasis web yang dirancang menggunakan metode *Scrum*, menerapkan konsep *CRUD* terintegrasi berbasis RDBMS, serta mendukung pemrosesan data secara *real-time*. Sistem ini dirancang dengan struktur basis data yang ternormalisasi untuk menjaga integritas data dan diuji secara menyeluruh menggunakan pendekatan *black box testing* untuk menjamin kualitas fungsionalnya (Nabella et al. 2025).

Berbeda dengan sebagian besar penelitian sebelumnya yang memfokuskan sistem peminjaman sarana dan prasarana pada interaksi administratif antara pengguna dan pengelola, seperti pada sistem manajemen aset sekolah (Nabella et al. 2025) maupun sistem manajemen ruang rapat (Herlambang et al. 2020), penelitian ini juga mengintegrasikan peran petugas keamanan (*security*) sebagai aktor sistem untuk melakukan validasi kehadiran peminjam secara langsung di lapangan. Integrasi ini memungkinkan sistem tidak hanya mencatat dan menyetujui peminjaman secara digital, tetapi juga memastikan kesesuaian antara jadwal peminjaman pada sistem dengan penggunaan sarana secara aktual guna meminimalkan risiko ketidakakuratan data dan konflik jadwal (Purwati, Pratama, and Rapiyanta 2022). Pendekatan ini bertujuan untuk menghilangkan celah verifikasi operasional (*operational blind spot*) yang sering terjadi pada sistem peminjaman konvensional, serta memperkuat pengendalian pemanfaatan sarana dan prasarana kampus secara menyeluruh (Ramdany and Bahari 2024).

Kontribusi utama dari penelitian ini meliputi: (1) penyajian implementasi metode *Scrum* secara sistematis dalam pengembangan sistem informasi peminjaman sarana dan prasarana kampus berbasis web yang terintegrasi dengan validasi administratif dan lapangan; (2) perancangan dan implementasi sistem *CRUD* terintegrasi dan *real-time* yang mendukung efisiensi operasional dengan pendekatan *green computing*; (3) perancangan struktur basis data ternormalisasi untuk meminimalkan redundansi dan meningkatkan kualitas pengelolaan data; serta (4) evaluasi kualitas sistem melalui kombinasi pengujian fungsional dan pengukuran *code coverage*. Diharapkan hasil penelitian ini dapat menjadi referensi dan solusi praktis bagi institusi pendidikan dalam mengembangkan sistem pengelolaan sarana dan prasarana kampus yang efektif, efisien, dan berkelanjutan.

1.2 Rumusan Masalah

Berdasarkan latar belakang permasalahan yang telah diuraikan pada bagian sebelumnya, pengelolaan peminjaman sarana dan prasarana kampus masih

menghadapi berbagai kendala, seperti proses yang belum terintegrasi, potensi redundansi data, keterbatasan pemantauan peminjaman secara *real-time*, serta belum optimalnya penerapan metode pengembangan dan pengujian sistem yang sistematis. Oleh karena itu, diperlukan suatu sistem informasi berbasis web yang dirancang dengan pendekatan pengembangan yang adaptif, efisien, dan berorientasi pada kualitas sistem.

Adapun rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana penerapan metode *Scrum* dalam perancangan dan pengembangan sistem informasi peminjaman sarana dan prasarana kampus berbasis web?
2. Bagaimana perancangan dan implementasi sistem informasi peminjaman sarana dan prasarana kampus berbasis web menggunakan *CRUD* terintegrasi dan *real-time* dengan penerapan *green computing* untuk meningkatkan efisiensi sistem?
3. Bagaimana merancang struktur basis data menggunakan teknik normalisasi untuk meminimalkan redundansi data pada sistem informasi peminjaman sarana dan prasarana kampus?
4. Bagaimana penerapan metode pengujian *black box testing* dan pengukuran *code coverage* dalam mengevaluasi kualitas sistem?

1.3 Tujuan

Tujuan dari penelitian ini adalah untuk merancang, mengembangkan, dan mengevaluasi sistem informasi peminjaman sarana dan prasarana kampus berbasis web yang efektif, efisien, dan berkualitas melalui penerapan metode pengembangan dan pengujian sistem yang sistematis.

Secara khusus, tujuan penelitian ini adalah sebagai berikut:

1. Menerapkan metode *Scrum* dalam proses perancangan dan pengembangan sistem informasi peminjaman sarana dan prasarana kampus berbasis web secara iteratif dan adaptif.
2. Merancang dan mengimplementasikan sistem informasi peminjaman sarana dan prasarana kampus berbasis web menggunakan konsep *CRUD* terintegrasi dan *real-time* dengan penerapan prinsip *green computing* untuk meningkatkan efisiensi penggunaan sumber daya sistem.

3. Merancang struktur basis data ternormalisasi guna meminimalkan redundansi data serta menjaga konsistensi dan integritas data pada sistem informasi peminjaman sarana dan prasarana kampus.
4. Mengevaluasi kualitas sistem yang dikembangkan melalui penerapan metode *Black Box Testing* untuk validasi fungsionalitas, serta pengukuran *Code Coverage* menggunakan fitur `go test -cover` untuk menganalisis efektivitas pengujian unit dan tingkat keterujian kode sumber secara mendalam.

1.4 Lingkup Dokumentasi

Ruang lingkup dokumentasi pada penelitian ini mencakup seluruh tahapan perancangan, implementasi, hingga evaluasi Sistem Informasi Peminjaman Sarana dan Prasarana berbasis web yang diterapkan di lingkungan kampus Universitas Logistik dan Bisnis Internasional (ULBI). Pengembangan sistem dilakukan dengan menerapkan metode *Scrum* guna memastikan proses pengembangan yang adaptif dan kolaboratif.

Secara spesifik, batasan dan ruang lingkup dokumentasi dalam penelitian ini meliputi:

1. Analisis dan perancangan sistem, yang mencakup analisis kebutuhan fungsional dan non-fungsional untuk mendukung konsep *CRUD* terintegrasi, pemrosesan data secara *real-time*, serta penerapan prinsip *green computing* dalam efisiensi penggunaan sumber daya sistem.
2. Implementasi fungsionalitas utama sistem, meliputi pengelolaan data sarana dan prasarana, mekanisme pengajuan dan persetujuan peminjaman, sistem penjadwalan, serta pencatatan status peminjaman secara digital.
3. Penerapan metodologi pengembangan *Scrum*, yang mencakup penyusunan *product planning*, *product backlog*, *sprint planning*, *sprint execution*, *sprint review*, dan *sprint retrospective*.
4. Perancangan arsitektur data dan antarmuka pengguna, yang meliputi perancangan struktur basis data relasional menggunakan teknik normalisasi untuk menjaga integritas data serta implementasi antarmuka pengguna (*user interface*) dengan memperhatikan aspek *usability*.
5. Evaluasi dan pengujian kualitas sistem, yang dilakukan melalui penerapan metode *Black Box Testing* untuk memverifikasi kesesuaian fungsionalitas sistem serta pengukuran *code coverage* menggunakan fitur `go test -cover` sebagai indikator kualitas dan tingkat keterujian kode program.

BAB 2

LANDASAN TEORI

2.1 Tinjauan Pustaka

Penelitian mengenai pengembangan sistem informasi manajemen aset dan peminjaman telah banyak dilakukan untuk mengatasi permasalahan operasional di institusi pendidikan maupun pemerintahan. Purwati, Pratama, and Rapiyanta (2022) mengembangkan sistem informasi peminjaman peralatan jaringan dan multimedia menggunakan metode *Waterfall*. Sistem ini dibangun berbasis *website* dengan bahasa pemrograman PHP dan basis data MySQL. Kelebihan penelitian ini adalah keberhasilannya dalam menggantikan pencatatan manual yang rentan hilang, namun penggunaan metode *Waterfall* dinilai kurang fleksibel terhadap perubahan kebutuhan pengguna yang terjadi di tengah proses pengembangan karena sifatnya yang sekuensial.

Dalam konteks manajemen sarana yang lebih luas, Nabella et al. (2025) melakukan penelitian terkait sistem manajemen sarana dan prasarana (Sarpras), IT, serta laboratorium dengan menggunakan metode *Rational Unified Process* (RUP). Sistem ini dikembangkan menggunakan *framework* CodeIgniter 4 dan MySQL untuk memusatkan data aset yang sebelumnya tersebar dalam file Excel. Penelitian ini unggul dalam standarisasi dokumentasi dan integrasi data untuk keperluan audit. Meskipun demikian, pendekatan RUP yang digunakan cenderung berat pada dokumentasi dan kurang adaptif dibandingkan pendekatan *Agile* dalam merespons umpan balik pengguna secara cepat selama fase pengembangan.

Terkait penerapan metodologi yang lebih adaptif, Rifky, Mursityo, and Prakoso (2022) menerapkan kerangka kerja *Scrum* dalam pengembangan sistem informasi pendataan bangunan. Penelitian ini menggunakan PHP dan *framework* CodeIgniter dengan arsitektur MVC. Hasil studi menunjukkan bahwa *Scrum* efektif diterapkan pada proyek dengan batasan waktu yang ketat dan mampu mengakomodasi perubahan fitur yang dinamis melalui mekanisme *sprint*. Sejalan dengan itu, Velasquez-Jimenez et al. (2025) juga mengimplementasikan perangkat lunak manajemen inventaris menggunakan metodologi *Scrum* yang diuji menggunakan *System Usability Scale* (SUS), yang membuktikan bahwa pendekatan iteratif meningkatkan kepuasan pengguna dan akurasi data inventaris.

Selain aspek metodologi, aspek kinerja teknis *backend* juga menjadi fokus dalam pengembangan sistem modern. Pratama and Farisi (2025) melakukan analisis komparatif kinerja RESTful API menggunakan bahasa pemrograman Go, PHP, dan JavaScript dengan metode Raw SQL dan ORM. Temuan penelitian ini menunjukkan bahwa penggunaan bahasa Go dengan metode Raw SQL memberikan kinerja tertinggi dalam menangani beban permintaan dan efisiensi memori dibandingkan PHP. Hal ini mengindikasikan bahwa untuk sistem yang membutuhkan pemrosesan data cepat dan *real-time*, pemilihan teknologi *backend* sangat krusial, aspek yang sering kali kurang diperhatikan dalam penelitian sistem informasi peminjaman sebelumnya yang mayoritas masih berbasis PHP konvensional.

Berdasarkan paparan penelitian terdahulu, mayoritas pengembangan sistem informasi peminjaman sarana prasarana masih didominasi oleh penggunaan metode tradisional seperti *Waterfall* atau RUP, yang memiliki keterbatasan dalam fleksibilitas adaptasi kebutuhan. Di sisi lain, meskipun metode *Scrum* telah diterapkan pada sistem pendataan bangunan dan inventaris, penerapannya secara spesifik pada sistem peminjaman sarana kampus yang mengintegrasikan konsep *CRUD* RDBMS yang optimal, pemrosesan *real-time*, serta prinsip *green computing* masih belum banyak dieksplorasi. Penelitian ini hadir untuk mengisi celah tersebut (*research gap*) dengan mengusulkan pengembangan sistem informasi peminjaman sarana dan prasarana berbasis web menggunakan metode *Scrum* untuk menjamin fleksibilitas pengembangan, serta mengadopsi teknologi yang mendukung efisiensi sumber daya dan kinerja tinggi guna mendukung operasional kampus yang berkelanjutan.

2.2 Sistem Informasi dan Manajemen Sarana Prasarana

Sistem informasi merupakan sistem terintegrasi yang mengombinasikan manusia, perangkat keras, perangkat lunak, jaringan, dan basis data untuk mengolah serta menyajikan informasi guna mendukung operasional dan pengambilan keputusan dalam organisasi (Herlambang et al. 2020). Dalam konteks perguruan tinggi, sistem informasi menjadi instrumen utama transformasi digital untuk menggantikan proses pengelolaan sarana dan prasarana yang masih manual atau semi-digital, sehingga informasi dapat disajikan secara cepat, akurat, dan konsisten (Purwati, Pratama, and Rapiyanta 2022).

Sarana dan prasarana kampus merupakan aset pendukung penting bagi kegiatan akademik dan non-akademik, sehingga pengelolaannya memerlukan manajemen yang sistematis. Proses peminjaman sarana dan prasarana umumnya mencakup tahapan pengajuan, verifikasi ketersediaan, persetujuan, penjadwalan, hingga pengembalian

aset. Dalam konteks operasional kampus, proses peminjaman tersebut juga melibatkan peran petugas keamanan sebagai pihak yang memastikan kesesuaian antara jadwal peminjaman pada sistem dengan penggunaan sarana secara aktual di lapangan. Implementasi proses tersebut dalam sistem informasi berbasis web memungkinkan integrasi data inventaris dan transaksi peminjaman secara terpusat serta mendukung pemantauan ketersediaan fasilitas secara *real-time* (Nabella et al. 2025).

Pengelolaan sarana dan prasarana yang masih dilakukan secara manual atau menggunakan *spreadsheet* terpisah berpotensi menimbulkan berbagai permasalahan, seperti redundansi data, keterlambatan pelaporan, dan risiko bentrok jadwal peminjaman akibat keterbatasan informasi ketersediaan fasilitas (Herlambang et al. 2020). Oleh karena itu, diperlukan sistem informasi peminjaman sarana dan prasarana kampus yang terintegrasi sebagai landasan perancangan sistem pada penelitian ini.

2.3 Metodologi *Scrum*

Pengembangan sistem informasi peminjaman sarana dan prasarana kampus menuntut metode yang mampu beradaptasi terhadap perubahan kebutuhan pengguna selama proses transformasi dari sistem manual ke digital. Berdasarkan penelitian terdahulu, kerangka kerja *Scrum* terbukti efektif dalam pengembangan sistem dengan kebutuhan dinamis dan batasan waktu yang ketat karena mendukung pengembangan bertahap serta validasi berulang pada setiap iterasi (Rifky, Mursityo, and Prakoso 2022; Velasquez-Jimenez et al. 2025). Oleh karena itu, *Scrum* dinilai relevan untuk digunakan dalam penelitian ini guna meminimalkan risiko ketidaksesuaian fungsi sistem dengan kebutuhan operasional pengguna.

Scrum didefinisikan sebagai kerangka kerja *Agile* yang bersifat iteratif dan inkremental untuk menghasilkan solusi adaptif terhadap permasalahan kompleks melalui transparansi, inspeksi, dan adaptasi (Schwaber and Sutherland 2020). Dalam penerapannya, *Scrum* melibatkan peran utama *Product Owner*, *Scrum Master*, dan *Development Team*, serta didukung oleh artefak *Product Backlog*, *Sprint Backlog*, dan *Increment* yang dikelola melalui serangkaian event seperti *Sprint Planning*, *Daily Scrum*, *Sprint Review*, dan *Sprint Retrospective* (Permana 2015; Nugraha and Hardian 2021). Karakteristik tersebut menjadikan *Scrum* sebagai landasan metodologis yang sesuai untuk pengembangan sistem peminjaman sarana kampus secara modular dan adaptif, yang pada penelitian ini akan diimplementasikan melalui mekanisme *sprint* pada tahap perancangan dan pengembangan sistem di Bab 3.

2.4 Basis Data Relasional dan Konsep *CRUD*

Sistem peminjaman sarana dan prasarana kampus melibatkan pengelolaan data yang kompleks, mencakup data pengguna, inventaris aset, serta jadwal pemakaian yang bersifat dinamis. Pengelolaan data yang masih dilakukan secara manual atau menggunakan *spreadsheet* terpisah sering menimbulkan permasalahan berupa data yang tidak terintegrasi, kesulitan penelusuran riwayat transaksi, serta keterlambatan penyusunan laporan. Kondisi serupa juga ditemukan pada berbagai institusi pendidikan, di mana keterbatasan sistem pencatatan konvensional berdampak pada rendahnya efisiensi dan akurasi pengelolaan sarana dan prasarana (Nabella et al. 2025).

Pendekatan yang umum digunakan untuk mengatasi permasalahan tersebut adalah penerapan *Relational Database Management System* (RDBMS), yaitu sistem manajemen basis data yang mengorganisasikan data ke dalam tabel-tabel yang saling berelasi melalui kunci utama (*primary key*) dan kunci asing (*foreign key*). Struktur relasional memungkinkan integrasi data secara terpusat, menjaga konsistensi informasi, serta mendukung akses data lintas unit kerja secara *real-time*, sebagaimana diterapkan pada pengembangan sistem manajemen inventaris dan peminjaman di lingkungan pendidikan (Velasquez-Jimenez et al. 2025; Ramdany and Bahari 2024). Interaksi antara aplikasi dan basis data dalam sistem informasi dilakukan melalui mekanisme *CRUD* (*Create, Read, Update, Delete*), yang merepresentasikan siklus hidup data mulai dari pencatatan aset, pemrosesan peminjaman, hingga pembaruan dan penghapusan data yang tidak lagi relevan (Purwati, Pratama, and Rapiyanta 2022).

Untuk menjaga kinerja dan kualitas basis data, diperlukan penerapan teknik normalisasi basis data guna meminimalkan redundansi dan mencegah terjadinya anomali data. Struktur basis data yang ternormalisasi memungkinkan setiap atribut disimpan secara tepat sesuai ketergantungannya, sehingga integritas data tetap terjaga dan efisiensi penyimpanan meningkat. Integrasi antara RDBMS, operasi *CRUD* yang efisien, serta struktur data yang ternormalisasi menjadi landasan teoretis utama dalam perancangan arsitektur data pada penelitian ini, yang implementasi teknisnya akan dibahas lebih lanjut pada Bab 3, sejalan dengan kebutuhan sistem peminjaman yang andal dan berkinerja baik (Pratama and Farisi 2025).

2.5 Teknologi Sistem Informasi Berbasis Web dengan Prinsip *Green Computing*

Pengembangan sistem informasi peminjaman sarana dan prasarana kampus memerlukan platform yang mampu mendukung interaksi *multi-user* secara terpusat dengan akses informasi ketersediaan aset secara *real-time* guna meminimalkan bentrok

jadwal dan kesalahan pencatatan. Sistem juga harus mengakomodasi akses lintas peran (*role-based access*), mulai dari mahasiswa hingga administrator, untuk menjamin keamanan dan keteraturan alur kerja. Oleh karena itu, teknologi berbasis web dipilih karena mampu mengintegrasikan seluruh proses peminjaman ke dalam satu sistem terpusat yang meningkatkan akurasi data dan efisiensi pelaporan dibandingkan metode manual atau penggunaan *spreadsheet* terpisah (Herlambang et al. 2020; Nabella et al. 2025; Velasquez-Jimenez et al. 2025).

Secara konseptual, aplikasi web beroperasi menggunakan arsitektur *client-server* yang memungkinkan pemisahan antara antarmuka pengguna dan pemrosesan data di sisi server melalui protokol HTTP. Arsitektur ini menawarkan keunggulan berupa aksesibilitas tinggi, kemudahan integrasi dengan sistem administratif kampus, serta skalabilitas yang baik dalam menangani peningkatan jumlah pengguna (Purwati, Pratama, and Rapiyanta 2022; Ramdany and Bahari 2024). Kinerja sistem web sangat bergantung pada komponen *backend* dan pengelolaan basis data dalam menangani logika bisnis serta operasi *CRUD*, di mana efisiensi pemrosesan dan pemilihan metode pengelolaan data yang tepat berperan penting dalam menjaga stabilitas dan waktu respons sistem, terutama pada beban akses yang tinggi (Pratama and Farisi 2025).

Sejalan dengan tuntutan keberlanjutan global, pengembangan sistem informasi modern juga harus mengadopsi prinsip efisiensi sumber daya atau *Green Computing*. *Green Computing* atau *Green IT* mencakup praktik perancangan, penggunaan, dan pembuangan sistem komputer dengan dampak lingkungan seminimal mungkin, baik melalui efisiensi energi maupun pengurangan limbah (Murugesan and Gangadharan 2012). Implementasi nyata konsep ini dalam sistem peminjaman adalah penerapan *paperless office*, yang mendigitalkan formulir dan laporan untuk mengurangi konsumsi kertas dan emisi karbon secara signifikan (Kim et al. 2021). Lebih jauh lagi, prinsip ini juga diterapkan pada level desain perangkat lunak, di mana algoritma dan kode program dioptimalkan untuk meminimalkan beban kerja prosesor dan memori, sehingga mengurangi konsumsi energi operasional server (Pratama and Farisi 2025).

2.6 Pengujian Perangkat Lunak

Pengujian perangkat lunak merupakan tahapan fundamental dalam siklus pengembangan sistem untuk menjamin kebenaran, kelengkapan, dan kualitas produk akhir. Taley and Pathak (2020) menjelaskan bahwa pengujian tidak hanya bertujuan untuk menemukan kesalahan (*bugs*), tetapi juga memverifikasi dan memvalidasi bahwa perangkat lunak telah memenuhi persyaratan teknis serta kebutuhan pengguna

sebelum dirilis. Dalam konteks pengembangan sistem informasi manajemen, pengujian yang sistematis diperlukan untuk memastikan bahwa alur kerja yang kompleks, seperti proses peminjaman dan validasi data, dapat berjalan tanpa hambatan dan risiko kegagalan fungsi dapat diminimalisir (Luh et al. 2020).

Salah satu metode utama yang digunakan untuk memvalidasi fungsionalitas sistem adalah *Black Box Testing*. Metode ini berfokus pada pengujian spesifikasi fungsional perangkat lunak tanpa perlu mengetahui struktur kode internal atau logika pemrograman di dalamnya (Luh et al. 2020). Pengujian ini dilakukan dengan memberikan serangkaian input pada antarmuka sistem dan mengevaluasi kesesuaian output yang dihasilkan dengan hasil yang diharapkan (Nabella et al. 2025). Pendekatan ini sangat relevan untuk sistem peminjaman sarana kampus guna memastikan bahwa fitur-fitur kritis, seperti formulir pengajuan, validasi stok aset, autentikasi pengguna, serta skenario validasi penggunaan sarana di lapangan oleh pihak terkait, berfungsi dengan benar sesuai skenario penggunaan (*use case*) yang telah dirancang (Purwati, Pratama, and Rapiyanta 2022).

Selain validasi fungsional, evaluasi kualitas teknis pada sisi *backend* dilakukan menggunakan pengukuran *Code Coverage*. *Code Coverage* adalah metrik yang digunakan untuk mengukur seberapa banyak baris kode program yang telah dieksekusi selama proses pengujian otomatis (*unit testing*). Dalam pengembangan *backend* menggunakan bahasa pemrograman Go (Golang), pengujian dan pengukuran coverage didukung secara bawaan oleh pustaka *testing* dan perintah *go test* (Kommadi 2019). Penggunaan alat ini memungkinkan pengembang untuk mengidentifikasi bagian logika program yang belum teruji (*uncovered code*) dan memastikan bahwa fungsi-fungsi krusial, seperti operasi *CRUD* dan logika bisnis, memiliki tingkat keterujian yang tinggi sebelum integrasi sistem dilakukan.

Kombinasi antara *Black Box Testing* untuk validasi eksternal dan *Code Coverage* untuk validasi internal membentuk strategi pengujian yang komprehensif. Landasan teori ini menjadi dasar bagi pelaksanaan pengujian sistem yang hasil dan analisisnya akan dipaparkan secara mendetail pada Bab 4 Implementasi dan Pengujian, guna membuktikan bahwa sistem informasi peminjaman sarana dan prasarana yang dibangun telah memenuhi standar kualitas yang ditetapkan.

BAB 3

METODOLOGI PENELITIAN

3.1 Metode Penelitian

Penelitian ini tergolong dalam jenis penelitian terapan (*applied research*) di bidang rekayasa perangkat lunak, yang bertujuan untuk merancang dan membangun solusi teknis bagi permasalahan pengelolaan aset kampus. Kerangka kerja *Scrum* ditetapkan sebagai metode utama dalam penelitian ini, bukan hanya sebagai panduan teknis, melainkan sebagai strategi fundamental untuk menjawab rumusan masalah terkait inefisiensi dan ketidakakuratan data pada proses manual saat ini. Pendekatan *Scrum* yang iteratif dan inkremental memungkinkan pengembangan perangkat lunak dilakukan secara bertahap untuk menghasilkan luaran produk (*increment*) yang bernilai guna tinggi dan teruji kualitasnya dalam menyelesaikan kendala operasional yang spesifik (Velasquez-Jimenez et al. 2025).

Pemilihan *Scrum* didasarkan pada analisis konteks (*Context*) lingkungan pendidikan tinggi yang dinamis, di mana kebutuhan pengguna (*Population*)—mulai dari mahasiswa, staf, hingga petugas keamanan—sering kali berubah seiring berjalannya waktu. Sebagai bentuk intervensi (*Intervention*) terhadap sistem konvensional (*Comparison*) yang kaku dan rentan kesalahan, *Scrum* menawarkan fleksibilitas melalui tiga pilar utamanya: transparansi, inspeksi, dan adaptasi (Schwaber and Sutherland 2020). Pendekatan ini sangat relevan untuk mengatasi kompleksitas fitur sistem peminjaman sarana prasarana, seperti integrasi validasi keamanan dan pembaruan stok *real-time*, dengan memungkinkan tim pengembang merespons perubahan prioritas secara cepat untuk mencapai luaran (*Outcome*) berupa efisiensi operasional dan akurasi data yang optimal (Putrianasari et al. 2024).

Secara operasional, penelitian ini tidak dilakukan dalam satu siklus linear panjang, melainkan melalui serangkaian siklus pendek berulang yang disebut *Sprint*. Setiap *Sprint* mencakup tahapan perencanaan, implementasi, dan evaluasi yang menghasilkan fungsionalitas sistem yang siap pakai dan dapat divalidasi langsung oleh pengguna (Permana 2015). Sifat iteratif ini memungkinkan perbaikan berkelanjutan (*continuous improvement*) pada setiap akhir siklus, sehingga risiko ketidaksesuaian antara sistem yang dibangun dengan kebutuhan operasional kampus dapat dideteksi dan diperbaiki sedini mungkin, menjamin terciptanya sistem informasi yang efisien dalam menjawab tantangan pengelolaan aset (Rifky, Mursityo, and Prakoso 2022).

3.2 Kerangka Penelitian Berdasarkan *PICOC*

Untuk menjamin fokus penelitian yang terarah dan memiliki batasan operasional yang jelas, penelitian ini mengadopsi kerangka kerja *PICOC* (*Population, Intervention, Comparison, Outcome, Context*). Kerangka ini digunakan untuk memetakan strategi penyelesaian masalah dari proses manual yang diidentifikasi pada Bab 1 menuju solusi teknis yang akan dikembangkan (Putrianasari et al. 2024). Berikut adalah pemetaan elemen *PICOC* yang disesuaikan dengan kebutuhan spesifik pengembangan sistem di Universitas Logistik dan Bisnis Internasional (ULBI):

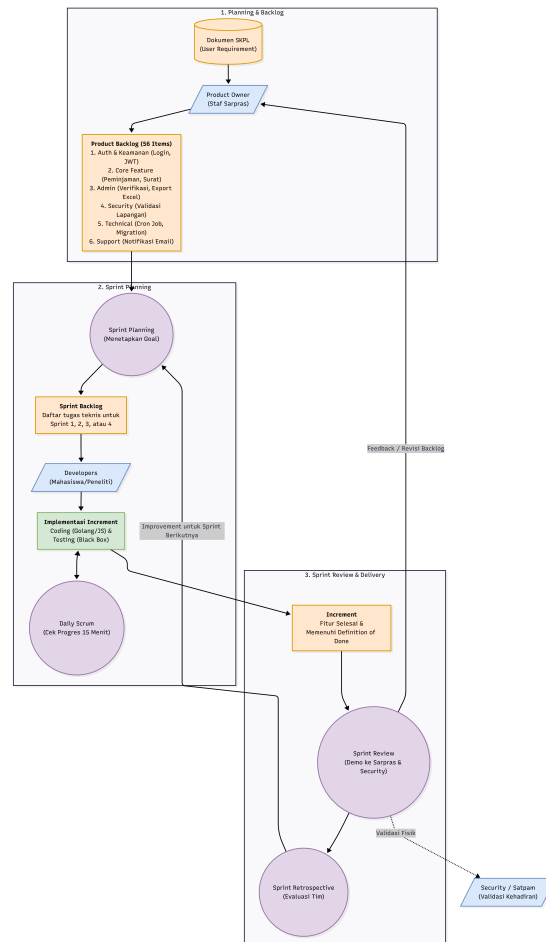
Tabel 3.1: Kerangka *PICOC* Penelitian

Kriteria	Definisi Operasional dalam Penelitian
Population	Civitas Akademika & Unit Pendukung: Mahasiswa, dan Staf Admin Sarpras sebagai pengguna utama, serta Petugas Keamanan (Security) sebagai aktor validator lapangan.
Intervention	Sistem Informasi Berbasis Web Terintegrasi: Penerapan teknologi web dengan backend Go dan basis data PostgreSQL yang menerapkan konsep CRUD RDBMS serta validasi ketersediaan secara <i>real-time</i> (Pratama and Farisi 2025; Velasquez-Jimenez et al. 2025).
Comparison	Proses Manual / Semi-Digital (As-Is): Metode pengelolaan saat ini yang berbasis formulir kertas, komunikasi lisan, dan arsip <i>spreadsheet</i> terpisah yang menyebabkan data tidak terintegrasi (<i>siloed data</i>) (Nabella et al. 2025; Purwati, Pratama, and Rapiyanta 2022).
Outcome	Efisiensi & Akurasi Operasional: Eliminasi redundansi data, pencegahan konflik jadwal (<i>double booking</i>), percepatan pelaporan, serta validasi penggunaan sarana di lapangan untuk menghilangkan celah keamanan aset (Ramdany and Bahari 2024).
Context	Lingkungan Kampus ULBI: Institusi pendidikan tinggi yang memerlukan transformasi digital untuk mendukung manajemen aset yang transparan dan prinsip efisiensi sumber daya (<i>Green Computing</i>) (Herlambang et al. 2020; Kim et al. 2021).

Penerapan kerangka *PICOC* di atas menegaskan bahwa intervensi teknologi yang dilakukan tidak hanya sekadar mendigitalkan formulir, melainkan mengubah alur kerja secara fundamental. Dengan membandingkan sistem berbasis web (*Intervention*) melawan proses manual (*Comparison*), penelitian ini menargetkan luaran (*Outcome*) spesifik berupa akurasi data yang divalidasi oleh petugas keamanan, sebuah fitur yang jarang ditemukan pada sistem konvensional. Konteks (*Context*) lingkungan ULBI menjadi batasan agar pengembangan fitur dalam metode *Scrum* tetap relevan dengan kebutuhan organisasi dan tidak meluas di luar cakupan manajemen sarana dan prasarana kampus (Putrianasari et al. 2024; Rifky, Mursityo, and Prakoso 2022).

3.3 Tahapan Pengembangan Sistem Menggunakan Metode *Scrum*

Alur penerapan metode *Scrum* dalam penelitian ini ditunjukkan pada Gambar 3.1, yang menggambarkan tahapan pengembangan sistem informasi peminjaman sarana dan prasarana kampus secara iteratif dan inkremental. Setiap tahapan pengembangan diturunkan dari permasalahan yang diidentifikasi pada Bab 1 dan dibatasi oleh kerangka *PICOC* pada Sub-bab 3.2 untuk memastikan relevansi solusi dengan kebutuhan pengguna serta konteks operasional Universitas Logistik dan Bisnis Internasional (ULBI) (Putrianasari et al. 2024). Proses pengembangan diawali dengan tahap *Planning* dan penyusunan *Product Backlog* berdasarkan dokumen spesifikasi kebutuhan pengguna (SKPL) yang dikelola oleh *Product Owner* (staf sarana dan prasarana), kemudian diprioritaskan dan diturunkan ke dalam *Sprint Backlog* melalui proses *Sprint Planning* (Schwaber and Sutherland 2020).



Gambar 3.1: Alur Penerapan Metode *Scrum*

Selanjutnya, *Sprint Execution* dilakukan oleh tim pengembang melalui implementasi dan pengujian awal yang menghasilkan *increment* sistem pada setiap iterasi, disertai koordinasi rutin melalui *Daily Scrum*. *Increment* yang dihasilkan dievaluasi pada tahap *Sprint Review* melalui demonstrasi sistem kepada staf sarana prasarana dan petugas keamanan untuk memastikan kesesuaian fungsionalitas, termasuk validasi penggunaan sarana secara fisik di lapangan. Proses pengembangan kemudian ditutup dengan *Sprint Retrospective* sebagai mekanisme evaluasi dan perbaikan berkelanjutan guna meningkatkan kualitas proses dan hasil pengembangan pada *sprint* berikutnya (Schwaber and Sutherland 2020).

3.3.1 Penetapan *Sprint Goal* dan *Product Backlog*

Tahap awal pengembangan dimulai dengan menetapkan *Sprint Goal* sebagai turunan dari tujuan utama penelitian, yaitu menciptakan sistem yang mampu menangani transaksi peminjaman secara efisien dan akurat. Berdasarkan tujuan tersebut, disusunlah *Product Backlog*, yaitu daftar prioritas kebutuhan sistem yang bersumber dari rumusan masalah penelitian dan hasil analisis kebutuhan pengguna (Permana 2015). Penyusunan backlog ini mempertimbangkan kebutuhan spesifik dari aktor-aktor yang terlibat (*Population*), seperti mahasiswa, staf administrasi, dan petugas keamanan, serta disesuaikan dengan aturan bisnis yang berlaku di lingkungan kampus ULBI (*Context*).

Item-item dalam *Product Backlog* dikelompokkan secara konseptual menjadi empat kategori utama untuk memudahkan manajemen pengembangan:

1. **Fondasi Sistem:** Mencakup manajemen basis data, autentikasi, dan hak akses pengguna.
2. **Fitur Inti Peminjaman:** Memfasilitasi alur transaksi pengajuan dan pengecekan ketersediaan dari sisi peminjam.
3. **Administrasi dan Validasi:** Berfokus pada kontrol persetujuan oleh staf sarana prasarana.
4. **Otomasi dan Notifikasi:** Mencakup fitur validasi lapangan oleh keamanan dan pelaporan otomatis.

3.3.2 Perencanaan *Sprint* (*Sprint Planning*)

Sprint Planning dilakukan pada awal setiap iterasi untuk menentukan backlog mana yang akan dieksekusi, dengan mempertimbangkan urgensi masalah, risiko operasional, dan ketergantungan antar fitur. Penentuan prioritas ini didasarkan pada

perbandingan (*Comparison*) antara proses manual yang ada dengan solusi yang diharapkan, di mana fitur yang berdampak langsung pada penghilangan hambatan operasional (seperti bentrok jadwal) dikerjakan lebih awal (Rifky, Mursityo, and Prakoso 2022). Tujuannya adalah memastikan efisiensi (*Outcome*) dapat segera terukur pada iterasi awal pengembangan.

3.3.3 Pelaksanaan *Sprint* (*Sprint Execution*)

Pada tahap *Sprint Execution*, tim pengembang mengimplementasikan fungsionalitas sistem sesuai dengan backlog yang telah disepakati. Pengembangan dilakukan secara bertahap di mana setiap *sprint* menghasilkan *increment* sistem yang berfungsi (*potentially shippable product*) (Schwaber and Sutherland 2020). Implementasi teknis pada tahap ini merupakan wujud nyata dari intervensi teknologi (*Intervention*) yang mengadopsi arsitektur berbasis web dengan penerapan konsep *CRUD* (*Create, Read, Update, Delete*) yang terintegrasi untuk mendukung pemrosesan data secara *real-time* (Pratama and Farisi 2025).

Rincian implementasi fitur yang dikerjakan dalam setiap iterasi disajikan dalam Tabel 3.2 berikut:

Tabel 3.2: Rincian Pelaksanaan *Sprint*

Tahap Iterasi	Fokus Pengembangan	Implementasi Fitur & Tujuan
Sprint 1	Fondasi Sistem & Data Master	Implementasi: Pembangun struktur basis data <i>RDBMS</i> , manajemen data aset (<i>CRUD</i>), dan modul autentikasi pengguna. Tujuan: Memastikan data aset tersimpan terstruktur dan hak akses pengguna terdistribusi dengan benar sebelum masuk ke fitur transaksi.
Sprint 2	Fitur Inti Peminjaman	Implementasi: Pengembangan formulir pengajuan digital, pengecekan ketersediaan jadwal, dan logika pencegahan <i>double booking</i> . Tujuan: Menggantikan proses pengajuan manual dan menyelesaikan masalah utama bentrok jadwal secara sistematis.
Sprint 3	Administrasi & Verifikasi	Implementasi: Pembuatan dasbor verifikasi untuk Admin Sarpras guna menyetujui atau menolak permohonan peminjaman. Tujuan: Memberikan kontrol administratif yang transparan dan <i>real-time</i> kepada pengelola aset.

Tahap Iterasi	Fokus Pengembangan	Implementasi Fitur & Tujuan
Sprint 4	Validasi Lapangan & Otomasi	Implementasi: Integrasi modul validasi kehadiran untuk Petugas Keamanan (Security) dan fitur pelaporan otomatis. Tujuan: Menutup celah keamanan aset (<i>operational blind spot</i>) dengan memvalidasi penggunaan fisik di lapangan sesuai data sistem.

3.3.4 Evaluasi *Sprint* (*Sprint Review*)

Sprint Review dilakukan di akhir setiap iterasi untuk meninjau dan memvalidasi hasil pengembangan (*Increment*) secara langsung bersama para pemangku kepentingan. Validasi fungsional ini melibatkan admin sarana prasarana dan petugas keamanan untuk memastikan bahwa fitur yang dibangun telah berjalan sesuai dengan kebutuhan pengguna (*Population*) (Velasquez-Jimenez et al. 2025). Evaluasi ini menjadi tolak ukur apakah sistem telah memenuhi kriteria keberhasilan (*Outcome*) yang ditetapkan dalam kerangka *PICOC*, khususnya terkait aspek efisiensi, akurasi data, dan keandalan validasi fisik di lapangan (Putrianasari et al. 2024).

3.3.5 Evaluasi Proses Pengembangan (*Sprint Retrospective*)

Sprint Retrospective bertujuan untuk mengevaluasi proses kerja tim *Scrum* itu sendiri, bukan hanya pada produk yang dihasilkan. Tahap ini berfokus pada identifikasi kendala yang muncul selama pelaksanaan *sprint*, baik dari sisi teknis maupun alur kerja, serta merumuskan langkah perbaikan untuk *sprint* berikutnya (Permana 2015). Penerapan prinsip perbaikan berkelanjutan (*continuous improvement*) ini sangat krusial untuk menjaga ritme pengembangan agar tetap adaptif terhadap dinamika kebutuhan penelitian dan memitigasi risiko kegagalan proyek (Putrianasari et al. 2024).

3.4 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk mengidentifikasi spesifikasi fungsional dan non-fungsional sebagai dasar perancangan sistem peminjaman sarana dan prasarana kampus. Analisis ini diturunkan dari Rumusan Masalah (Bab 1), batasan *PICOC* (Bab 3.2), serta *Sprint Goal* dalam metode *Scrum*, dengan tujuan memetakan *Product Backlog* secara terstruktur sebelum tahap perancangan dan implementasi, sehingga solusi yang dikembangkan mampu mengatasi inefisiensi proses manual, redundansi data, dan bentrok jadwal secara tepat.

3.4.1 Identifikasi Aktor Sistem

Berdasarkan elemen *Population* dalam kerangka *PICOC*, terdapat tiga aktor utama yang berinteraksi dengan sistem, masing-masing memiliki hak akses dan peran spesifik:

1. **Mahasiswa (Peminjam):** Aktor yang mengajukan permohonan peminjaman sarana, memantau status pengajuan, dan melihat ketersediaan jadwal secara mandiri.
2. **Admin Sarpras (Pengelola):** Aktor yang memiliki kewenangan penuh untuk mengelola data *master* aset, memverifikasi ketersediaan, serta menyetujui atau menolak pengajuan peminjaman berdasarkan aturan akademik.
3. **Petugas Keamanan (Validator):** Aktor khusus yang bertugas melakukan validasi kehadiran peminjam dan penggunaan sarana secara fisik di lapangan (*on-site validation*) untuk memastikan kesesuaian dengan data sistem.

3.4.2 Kebutuhan Fungsional Sistem

Kebutuhan fungsional mendefinisikan layanan-layanan spesifik yang harus disediakan oleh sistem untuk mendukung operasional harian. Daftar kebutuhan ini menjadi landasan dalam penyusunan *User Stories* pada *Product Backlog Scrum*:

1. **Autentikasi dan Otorisasi:** Sistem harus menyediakan mekanisme *login* yang aman dengan pembagian hak akses (*Role-Based Access Control*) untuk membedakan fitur antar aktor.
2. **Manajemen Data Aset (CRUD):** Admin Sarpras harus dapat melakukan operasi tambah, baca, ubah, dan hapus data sarana dan prasarana secara terpusat.
3. **Pengecekan Ketersediaan:** Sistem harus mampu menampilkan jadwal penggunaan ruangan dan alat secara *real-time* kepada pengguna untuk mencegah pengajuan pada *slot* waktu yang sudah terisi.
4. **Pengajuan Peminjaman:** Mahasiswa harus dapat mengisi formulir pengajuan digital yang mencakup detail kegiatan, waktu, dan aset yang dibutuhkan.
5. **Persetujuan Berjenjang:** Sistem harus memfasilitasi alur kerja verifikasi di mana pengajuan masuk dapat disetujui atau ditolak oleh Admin Sarpras dengan notifikasi status.
6. **Validasi Lapangan:** Sistem harus menyediakan antarmuka khusus bagi Petugas Keamanan untuk memvalidasi penggunaan aset saat kegiatan berlangsung.

7. **Pelaporan:** Sistem harus mampu menghasilkan rekapitulasi data peminjaman secara otomatis untuk keperluan audit manajemen.

3.4.3 Kebutuhan Non-Fungsional Sistem

Kebutuhan non-fungsional menetapkan batasan kualitas dan standar kinerja yang harus dipenuhi oleh sistem agar dapat beroperasi dengan optimal:

1. **Kinerja (*Performance*):** Sistem harus mampu memproses permintaan data secara *real-time* dengan waktu respons yang cepat, terutama saat pengecekan jadwal untuk menghindari *double booking*.
2. **Keamanan (*Security*):** Penerapan keamanan data meliputi enkripsi *password* (*hashing*), perlindungan terhadap serangan injeksi (*SQL Injection*), dan pembatasan akses data sensitif hanya untuk aktor yang berwenang.
3. **Ketersediaan (*Availability*):** Sistem harus dapat diakses 24/7 melalui jaringan internet kampus untuk mendukung fleksibilitas waktu pengajuan.
4. **Efisiensi Sumber Daya (*Green Computing*):** Sistem dirancang untuk mendukung prinsip nirkertas (*paperless*) dan efisiensi kode program guna meminimalkan beban komputasi server.
5. **Usability:** Antarmuka sistem harus dirancang ramah pengguna (*user-friendly*) sehingga mudah dipahami oleh pengguna awam tanpa memerlukan pelatihan intensif.

3.4.4 Aturan Bisnis (*Business Rules*)

Aturan bisnis mendefinisikan logika operasional kampus yang diadopsi ke dalam sistem untuk menjaga integritas proses bisnis. Aturan-aturan ini menjadi dasar logika validasi pada kode program:

1. **Integritas Jadwal:** Satu unit sarana atau prasarana tidak dapat dipinjam atau dipesan oleh dua pihak berbeda pada rentang waktu yang sama (pencegahan *double booking*).
2. **Mekanisme Persetujuan:** Setiap pengajuan peminjaman berstatus “*Pending*” dan belum sah digunakan sampai mendapatkan persetujuan digital (“*Approved*”) dari Admin Sarpras.
3. **Validasi Keamanan:** Penggunaan fasilitas fisik di lapangan hanya diizinkan jika peminjam dapat menunjukkan bukti persetujuan sistem yang kemudian divalidasi statusnya oleh Petugas Keamanan.

4. **Siklus Hidup Status:** Status peminjaman harus diperbarui secara otomatis atau manual sesuai siklus: Diajukan → Disetujui/Ditolak → Sedang Digunakan (Validasi *Security*) → Selesai.

3.5 Perancangan Sistem

Tahap perancangan sistem bertujuan untuk mentransformasikan hasil analisis kebutuhan menjadi representasi teknis yang siap diimplementasikan. Pada penelitian ini, perancangan dimulai dengan pemodelan arsitektur menggunakan Use Case Diagram untuk menggambarkan interaksi fungsional antara pengguna (aktor) dengan sistem. Pendekatan ini memastikan bahwa setiap fitur yang dibangun memiliki landasan logika operasional yang jelas sebelum diterjemahkan ke dalam kode program.

3.5.1 Usecase Diagram

Use Case Diagram merinci fungsionalitas sistem dari sudut pandang pengguna dengan memetakan hubungan antara aktor dan kasus penggunaan (*use case*). Berdasarkan rancangan yang ditunjukkan pada Gambar 3.2, sistem ini melibatkan tiga aktor utama yaitu Mahasiswa, Sarpras (Sarana dan Prasarana), dan *Security*, yang masing-masing memiliki hak akses spesifik terhadap fitur sistem.

Berikut adalah deskripsi interaksi aktor dalam sistem:

1. Mahasiswa:

- **Mengajukan Peminjaman:** Mahasiswa dapat melakukan permohonan peminjaman aset. Proses ini memiliki relasi *include* dengan *Upload Surat Digital*, yang berarti sistem mewajibkan unggah dokumen sebagai syarat mutlak pengajuan.
- **Melihat Jadwal Ruangan:** Mahasiswa dapat mengecek ketersediaan fasilitas untuk menghindari bentrok jadwal sebelum mengajukan peminjaman.

2. Sarpras (Administrator):

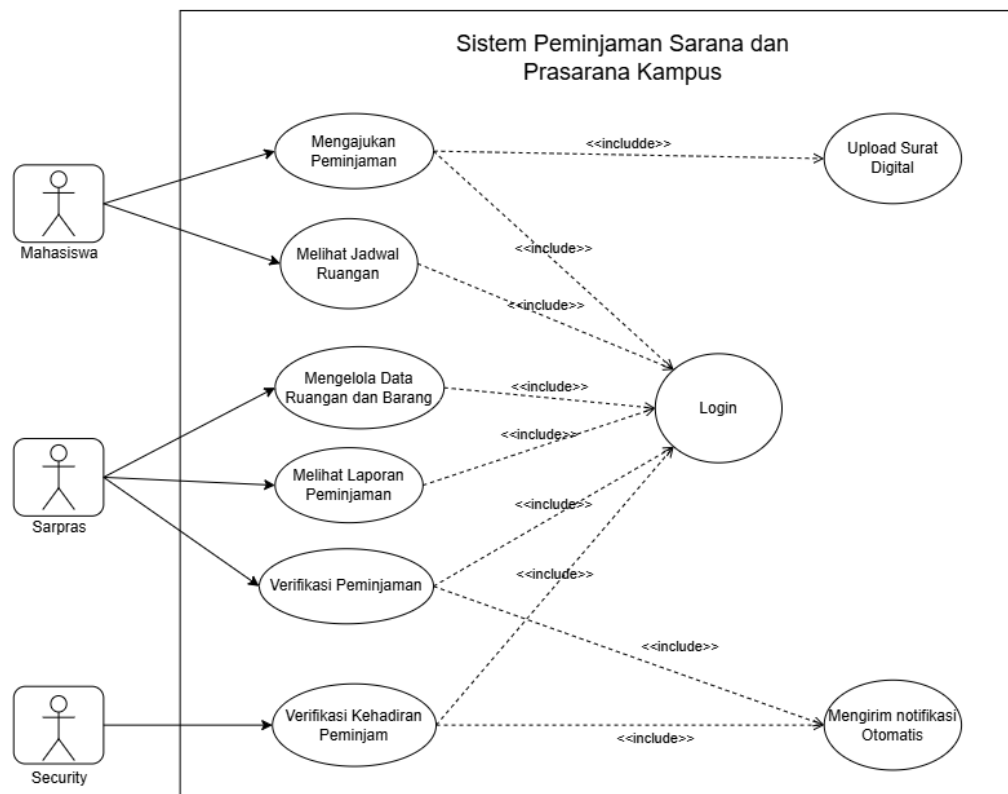
- **Mengelola Data Ruangan dan Barang:** Aktor ini bertanggung jawab penuh terhadap manajemen data *master inventaris (CRUD)*.
- **Verifikasi Peminjaman:** Sarpras bertugas menyetujui atau menolak pengajuan masuk. Aktivitas ini memiliki relasi *include* dengan Mengirim Notifikasi Otomatis, di mana sistem secara langsung memberikan kabar status kepada peminjam setelah verifikasi dilakukan.

- **Melihat Laporan Peminjaman:** Akses untuk memantau rekapitulasi aktivitas peminjaman.

3. Security (Petugas Keamanan):

- **Verifikasi Kehadiran Peminjam:** Aktor ini memvalidasi penggunaan fasilitas secara fisik di lapangan. Sama seperti verifikasi sarpras, fitur ini juga terhubung (*include*) dengan Mengirim Notifikasi Otomatis untuk memperbarui status peminjaman menjadi “sedang digunakan” atau “selesai”.

Seluruh aktivitas fungsional di atas (kecuali notifikasi otomatis yang dijalankan sistem) mensyaratkan semua aktor untuk melewati proses *Login* terlebih dahulu guna menjamin keamanan dan otentikasi akses.



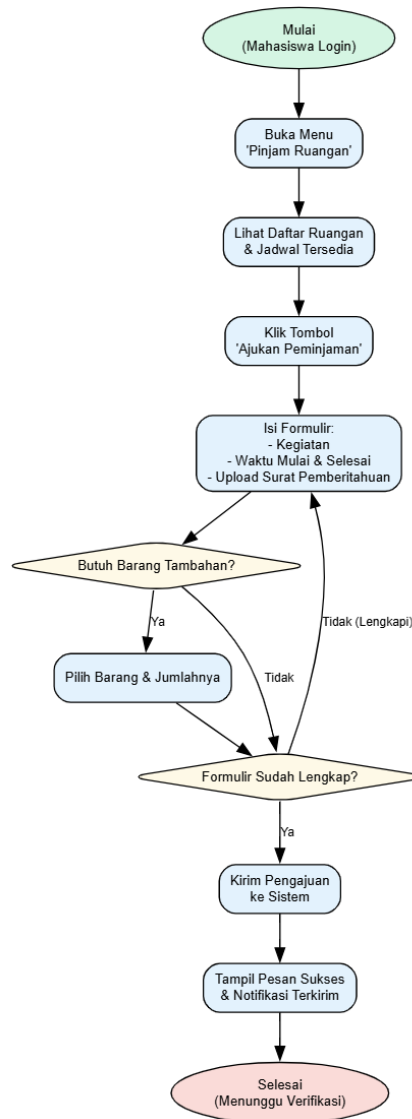
Gambar 3.2: Usecase Diagram Sistem

3.5.2 Flowchart Diagram

Flowchart atau diagram alur digunakan untuk menggambarkan urutan langkah-langkah logis dan keputusan dalam proses sistem. Berikut adalah penjabaran alur untuk proses utama dalam sistem:

1. Flowchart Diagram Alur Pengajuan Peminjaman

Flowchart alur pengajuan peminjaman menggambarkan langkah-langkah yang dilakukan oleh aktor Mahasiswa dalam mengajukan permohonan penggunaan sarana dan prasarana. Alur ini dirancang untuk memastikan kelengkapan data administratif sebelum permohonan diproses oleh sistem.



Gambar 3.3: Flowchart Diagram Alur Pengajuan Peminjaman

Berdasarkan rancangan alur yang ditunjukkan pada Gambar 3.3, proses dimulai ketika mahasiswa yang telah berhasil *login* mengakses menu “Pinjam Ruangan”. Sistem kemudian akan menampilkan daftar ruangan beserta informasi ketersediaan jadwal secara *real-time* untuk mencegah terjadinya bentrok jadwal (*double booking*). Mahasiswa kemudian memilih opsi “Ajukan Peminjaman” untuk memulai pengisian formulir digital.

Pada tahap pengisian formulir, mahasiswa diwajibkan melengkapi data detail kegiatan, waktu mulai dan selesai, serta mengunggah dokumen pendukung berupa Surat Pemberitahuan atau proposal kegiatan dalam format digital. Sistem menyediakan percabangan logika (*decision point*) untuk penambahan fasilitas pendukung; jika mahasiswa membutuhkan barang tambahan (seperti proyektor atau *sound system*), sistem akan mengarahkan pengguna untuk memilih jenis barang dan jumlahnya.

Sebelum data disimpan, sistem melakukan validasi otomatis untuk memeriksa kelengkapan formulir. Jika data belum lengkap, sistem akan meminta pengguna untuk melengkapinya kembali. Apabila seluruh data telah valid, pengajuan akan dikirim ke basis data dan sistem menampilkan pesan sukses serta notifikasi bahwa pengajuan telah terkirim. Proses berakhir dengan status peminjaman yang secara otomatis diatur menjadi “Menunggu Verifikasi” oleh bagian Sarana dan Prasarana.

2. Flowchart Diagram Alur Verifikasi Peminjaman

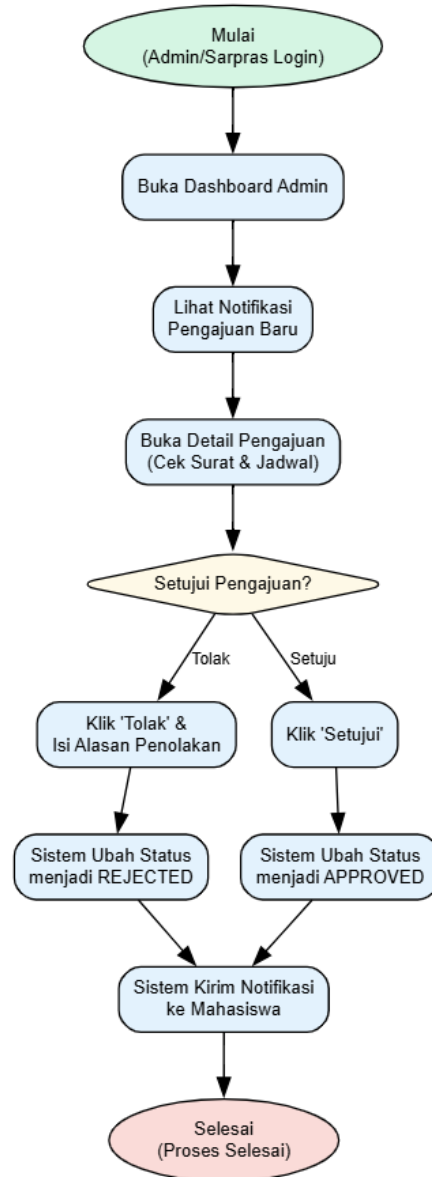
Flowchart verifikasi peminjaman menggambarkan urutan logika yang dilakukan oleh aktor Admin Sarpras dalam menindaklanjuti permohonan peminjaman yang telah diajukan oleh mahasiswa. Alur ini dirancang untuk menjamin validitas administrasi dan ketersediaan aset sebelum izin penggunaan diterbitkan.

Berdasarkan rancangan alur yang ditunjukkan pada Gambar 3.4, proses dimulai ketika Admin Sarpras berhasil masuk (*login*) ke dalam sistem dan mengakses *dashboard* admin. Sistem akan menampilkan Notifikasi Pengajuan Baru jika terdapat permohonan yang masuk. Admin kemudian membuka detail pengajuan untuk melakukan pemeriksaan manual terhadap kelengkapan dokumen (surat kegiatan) dan memastikan tidak ada bentrok jadwal pada sistem kalender.

Pada tahap ini terdapat titik keputusan (*decision point*) “Setujui Pengajuan?” yang membagi alur menjadi dua skenario:

1. **Jika Ditolak:** Admin menekan tombol “Tolak” dan diwajibkan mengisi Alasan Penolakan pada formulir. Sistem kemudian akan melakukan operasi *update* pada basis data untuk mengubah status peminjaman menjadi *REJECTED*.

2. **Jika Disetujui:** Admin menekan tombol “Setujui”. Sistem akan memproses persetujuan tersebut dan mengubah status peminjaman menjadi *APPROVED* dalam basis data.

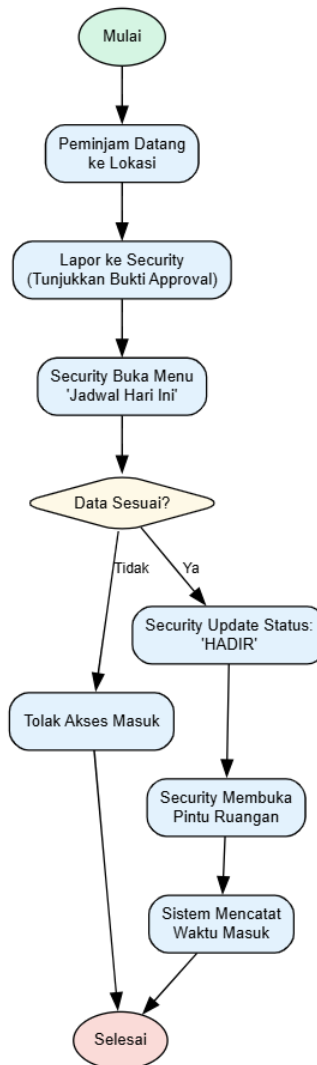


Gambar 3.4: *Flowchart Diagram Alur Verifikasi Peminjaman*

Setelah status diperbarui (baik disetujui maupun ditolak), sistem secara otomatis memicu fungsi untuk Mengirim Notifikasi kepada akun mahasiswa terkait status akhir permohonan mereka. Proses verifikasi dianggap selesai setelah status tersimpan dan notifikasi terkirim.

3. Flowchart Diagram Alur Verifikasi Kehadiran Peminjam

Flowchart verifikasi kehadiran menggambarkan prosedur validasi fisik yang dilakukan oleh aktor Petugas Keamanan (*Security*) di lapangan. Alur ini merupakan fitur krusial yang membedakan sistem ini dengan sistem konvensional, di mana validasi dilakukan untuk memastikan sinkronisasi antara data peminjaman di sistem dengan penggunaan aktual fasilitas.



Gambar 3.5: Flowchart Diagram Alur Verifikasi Kehadiran Peminjam

Berdasarkan rancangan alur yang ditunjukkan pada Gambar 3.5, proses dimulai ketika Peminjam Datang ke Lokasi dan melapor kepada petugas keamanan dengan menunjukkan bukti persetujuan (*approval*) yang diperoleh dari sistem. Petugas keamanan kemudian mengakses sistem melalui perangkat *mobile* atau pos jaga dan membuka menu “Jadwal Hari Ini” untuk memverifikasi klaim peminjam.

Pada tahap ini, terjadi proses pengambilan keputusan (*decision point*) “Data Sesuai?”:

1. **Jika Tidak Sesuai:** Apabila data peminjam tidak ditemukan pada jadwal hari itu atau statusnya belum disetujui, petugas keamanan berhak melakukan Tolak Akses Masuk, dan proses selesai.
2. **Jika Sesuai:** Apabila data valid, petugas keamanan melakukan tindakan pada sistem untuk *Update Status*: ‘HADIR’. Tindakan ini secara *real-time* mencatat waktu kedatangan peminjam ke dalam basis data. Selanjutnya, petugas secara fisik Membuka Pintu Ruang untuk peminjam.

Sistem secara otomatis akan merekam waktu masuk (*timestamp*) sebagai data historis penggunaan aset. Mekanisme ini menjamin bahwa setiap ruangan yang digunakan benar-benar tercatat secara digital, mendukung akurasi pelaporan penggunaan sarana kampus.

3.5.3 Sequence Diagram

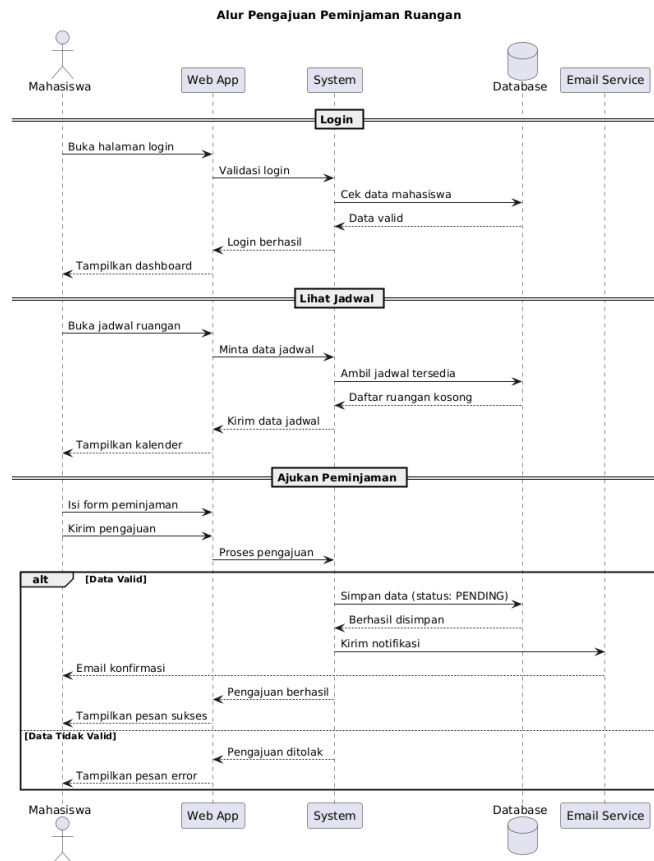
Sequence diagram memodelkan interaksi dinamis antar objek di dalam sistem berdasarkan urutan waktu. Diagram ini digunakan untuk merinci logika skenario penggunaan mulai dari antarmuka hingga pemrosesan di basis data. Berikut adalah penjabaran alur utama sistem:

1. Sequence Diagram Pengajuan Peminjaman

Diagram ini menggambarkan aliran pesan (*message flow*) antara aktor Mahasiswa dengan objek *Web App*, *System (Backend)*, *Database*, dan *Email Service*. Berdasarkan rancangan pada Gambar 3.6, proses ini terdiri dari tiga segmen logika utama:

1. **Otentikasi Akses:** Proses diawali dengan validasi kredensial login Mahasiswa. *System* memverifikasi data masukan terhadap *Database*; jika valid, hak akses diberikan dan *dashboard* ditampilkan.
2. **Validasi Ketersediaan:** Sebelum pengajuan, sistem mengambil data jadwal *real-time* dari *Database* dan menampilkannya pada antarmuka kalender. Langkah ini krusial untuk memastikan mahasiswa hanya memilih *slot* waktu yang tersedia guna mencegah *double booking*.
3. **Eksekusi Transaksi:** Saat formulir dikirim, *System* mengevaluasi kelengkapan data menggunakan blok logika alternatif (*alt*):

- (a) **[Data Valid]:** Data disimpan ke *Database* dengan status awal “*PENDING*”. Keberhasilan penyimpanan secara otomatis memicu *Email Service* untuk mengirimkan notifikasi konfirmasi ke mahasiswa.
- (b) **[Data Tidak Valid]:** Sistem membatalkan proses penyimpanan dan menginstruksikan *Web App* untuk menampilkan pesan kesalahan (*error message*) kepada pengguna.



Gambar 3.6: *Sequence Diagram* Pengajuan Peminjaman

2. *Sequence Diagram* Verifikasi Peminjaman

Diagram ini memvisualisasikan interaksi antara aktor Admin dengan objek *Web App*, *System (Backend)*, *Database*, dan *Email Service* dalam memproses persetujuan peminjaman. Berdasarkan alur pada Gambar 3.7, proses ini mencakup tiga tahapan logika utama:

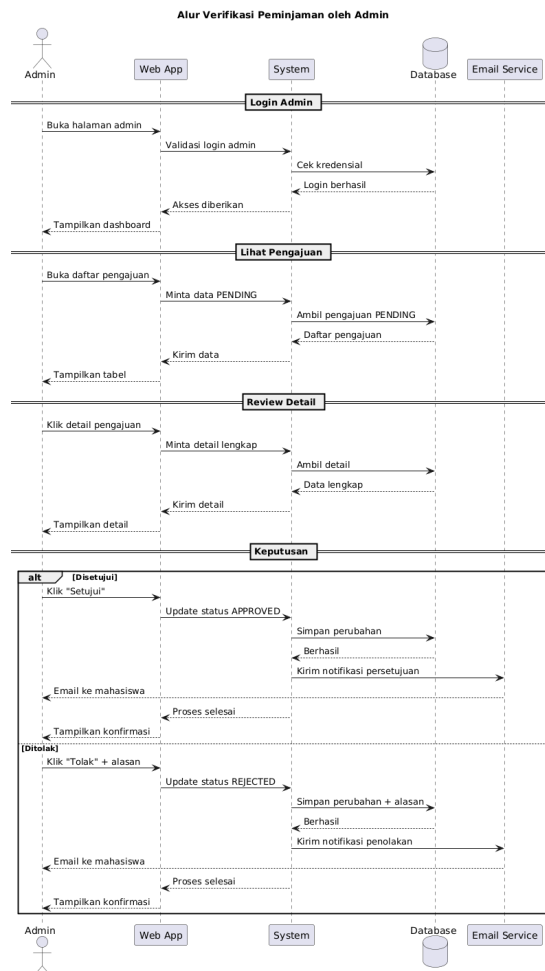
1. **Akses dan Peninjauan Data:** Setelah Admin berhasil login (validasi kredensial ke *Database*), *Web App* meminta data peminjaman berstatus “*PENDING*”. *System* mengambil data tersebut dari *Database* untuk ditampilkan kepada Admin guna peninjauan detail kegiatan dan dokumen pendukung.

2. **Eksekusi Keputusan (Approval/Rejection):** Admin memberikan keputusan yang ditangani oleh blok logika alternatif (*alt*):

(a) **[Disetujui]:** Admin menekan tombol setuju. *System* memperbarui status menjadi “*APPROVED*” di *Database*. Konfirmasi penyimpanan sukses memicu *Email Service* untuk mengirimkan notifikasi persetujuan ke mahasiswa.

(b) **[Ditolak]:** Admin menekan tombol tolak dan menyertakan alasan. *System* memperbarui status menjadi “*REJECTED*” di *Database* dan memicu *Email Service* untuk mengirim notifikasi penolakan beserta alasannya.

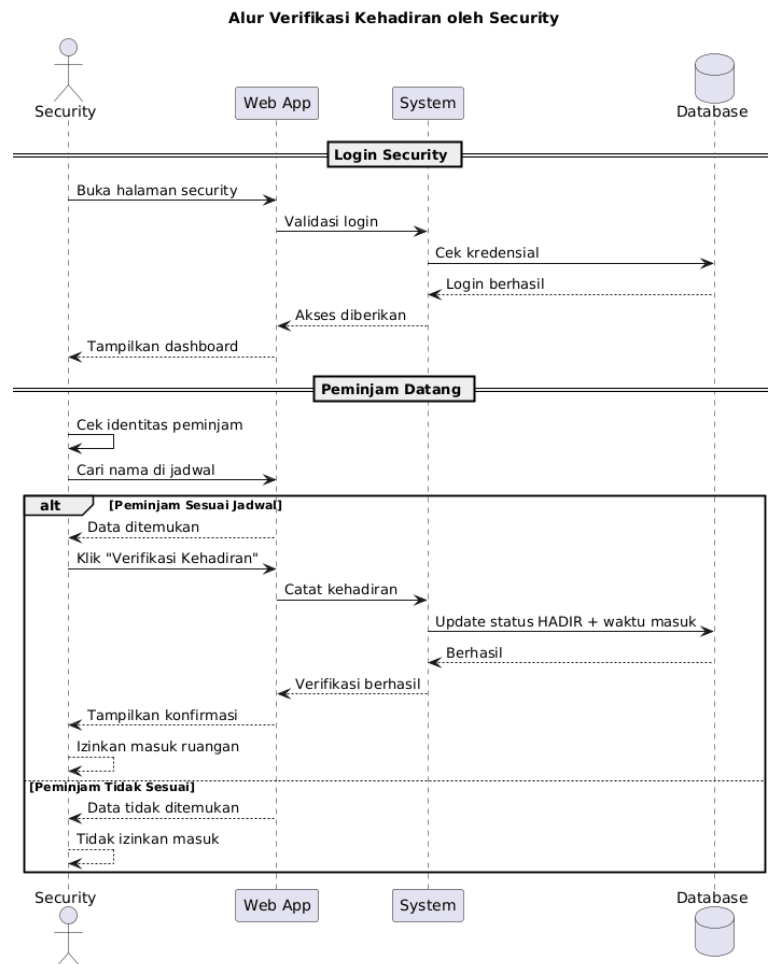
3. **Finalisasi:** Proses diakhiri dengan konfirmasi visual pada antarmuka Admin bahwa status telah berhasil diperbarui dan notifikasi telah terkirim.



Gambar 3.7: Sequence Diagram Verifikasi Peminjaman

3. Sequence Diagram Verifikasi Kehadiran

Diagram ini memvisualisasikan interaksi antara aktor Petugas Keamanan (*Security*) dengan objek *Web App*, *System (Backend)*, dan *Database* untuk memvalidasi penggunaan fasilitas secara fisik. Berdasarkan alur pada Gambar 3.8, proses ini mencakup tiga tahapan logika utama:



Gambar 3.8: Sequence Diagram Verifikasi Kehadiran

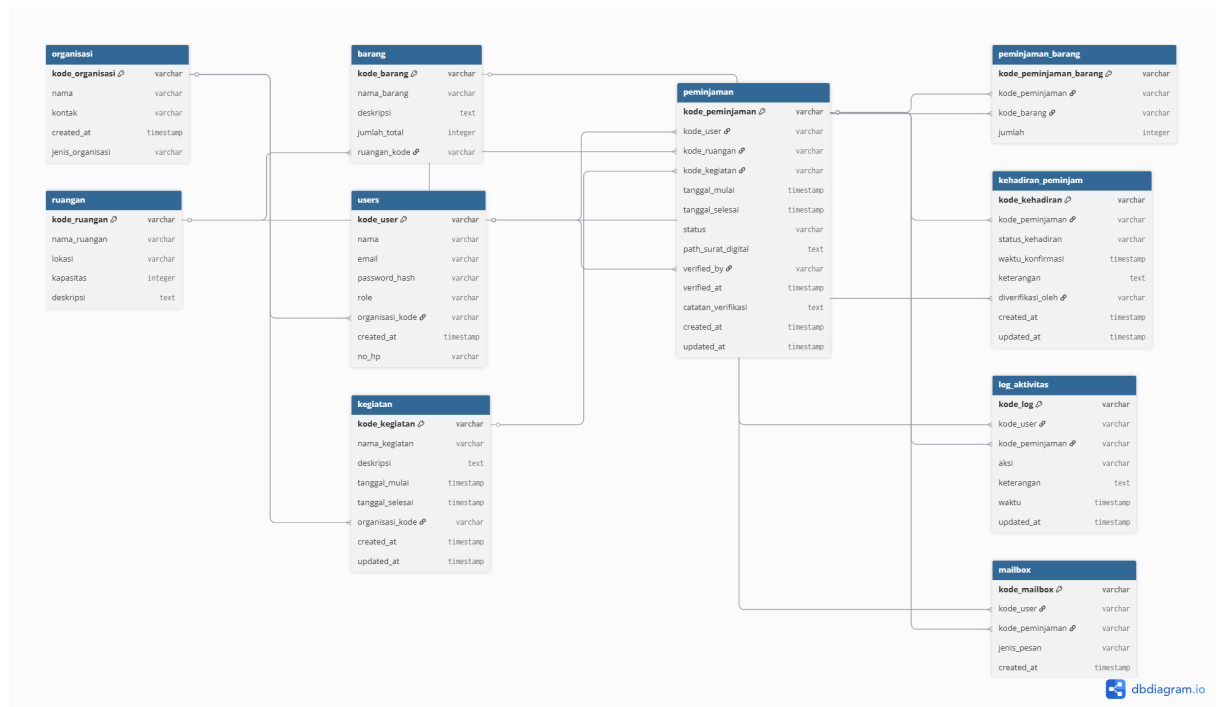
1. **Otentikasi Petugas:** Proses diawali dengan *Security* melakukan login pada aplikasi. *Web App* mengirimkan kredensial ke *System* untuk divalidasi terhadap *Database*. Setelah sukses, sistem menampilkan *dashboard* keamanan.
2. **Pengecekan Identitas:** Saat peminjam datang, *Security* memasukkan identitas atau memindai bukti peminjaman. *System* melakukan pencarian data (*query*) ke *Database* untuk mencocokkan jadwal dan status persetujuan.
3. **Eksekusi Validasi Lapangan:** Hasil pencarian ditangani oleh blok logika alternatif (*alt*):

- (a) **[Peminjam Sesuai Jadwal]:** Jika data valid, *Security* menekan tombol verifikasi. *System* mengirimkan perintah *update* ke *Database* untuk mengubah status menjadi “HADIR” dan mencatat waktu masuk (*timestamp*). Sistem memberikan konfirmasi sukses, dan *Security* mengizinkan akses masuk ruangan.
- (b) **[Peminjam Tidak Sesuai]:** Jika data tidak ditemukan atau jadwal salah, sistem menampilkan pesan “Data tidak ditemukan”, dan *Security* menolak akses masuk.

3.5.4 Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) digunakan untuk memodelkan struktur data dan hubungan antar entitas dalam basis data sistem. Perancangan ini bertujuan untuk memastikan integritas data dan efisiensi penyimpanan, mendukung konsep *RDBMS* (*Relational Database Management System*) yang menjadi fondasi teknis penelitian ini. Berdasarkan analisis kebutuhan, terdapat beberapa entitas utama yang saling berelasi untuk mendukung alur peminjaman dan validasi lapangan.

Sebagaimana diilustrasikan pada Gambar 3.9, skema basis data dirancang dengan hubungan relasional sebagai berikut:



Gambar 3.9: Entity Relationship Diagram (ERD)

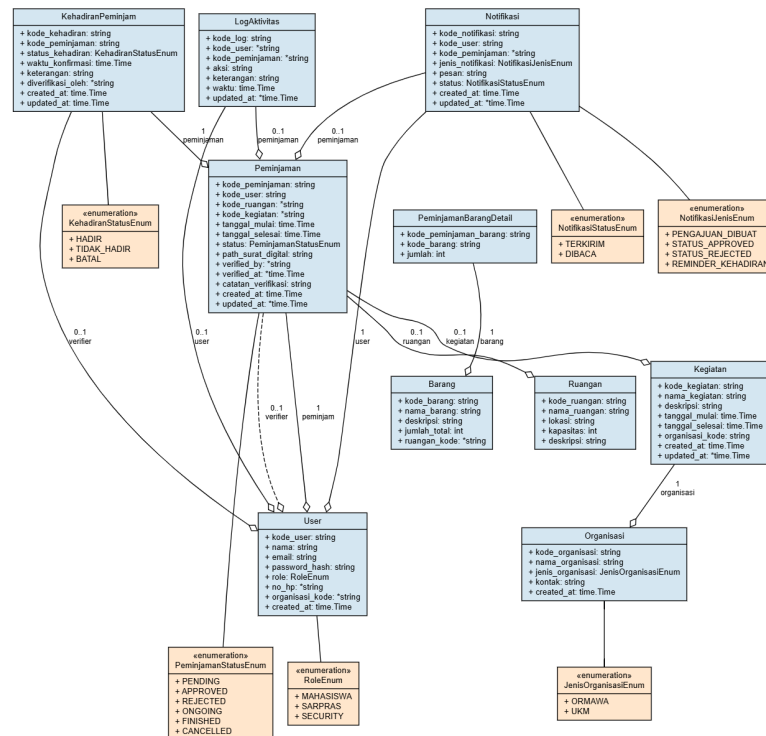
Dengan perancangan ERD yang terstruktur dan relasi antar entitas yang jelas sebagaimana ditunjukkan pada gambar ERD dan dirinci dalam Tabel 3.3, sistem ini mampu menjaga integritas data serta mendukung efisiensi operasi CRUD, proses verifikasi, dan pelaporan sebagai dasar implementasi basis data.

Tabel 3.3: Relasi Antar Entitas

No.	Entitas Utama	Entitas Terkait	Jenis Relasi	Keterangan
1	<i>users</i>	organisasi	<i>One-to-Many</i>	Satu organisasi dapat memiliki banyak pengguna
2	<i>users</i>	peminjaman	<i>One-to-Many</i>	Satu pengguna dapat mengajukan lebih dari satu peminjaman
3	ruangan	peminjaman	<i>One-to-Many</i>	Satu ruangan dapat digunakan dalam beberapa transaksi peminjaman
4	kegiatan	peminjaman	<i>One-to-Many</i>	Satu kegiatan dapat menjadi dasar beberapa peminjaman
5	peminjaman	peminjaman_barang	<i>One-to-Many</i>	Satu peminjaman dapat memiliki lebih dari satu barang
6	barang	peminjaman_barang	<i>One-to-Many</i>	Satu barang dapat tercatat pada banyak transaksi peminjaman
7	peminjaman	kehadiran_peminjam	<i>One-to-Many</i>	Setiap peminjaman memiliki satu data kehadiran
8	<i>users</i>	<i>mailbox</i>	<i>One-to-Many</i>	Satu pengguna dapat menerima banyak notifikasi
9	peminjaman	<i>mailbox</i>	<i>One-to-Many</i>	Satu peminjaman dapat menghasilkan banyak pesan
10	<i>users</i>	log_aktivitas	<i>One-to-Many</i>	Aktivitas pengguna tercatat dalam log sistem
11	peminjaman	log_aktivitas	<i>One-to-One</i>	Aktivitas terkait peminjaman tercatat dalam log

3.5.5 Class Diagram

Class Diagram menggambarkan struktur statis dari sistem dengan mendefinisikan kelas-kelas yang akan dibuat, atribut yang dimilikinya, serta hubungan antar kelas tersebut. Dalam konteks pengembangan menggunakan bahasa pemrograman Go (Golang), kelas-kelas ini direpresentasikan sebagai *Structs* yang mendefinisikan tipe data dan *Interfaces* yang mendefinisikan perilaku metode. Diagram ini berfungsi sebagai cetak biru (*blueprint*) teknis bagi pengembang dalam menulis kode program, memastikan bahwa logika bisnis seperti validasi kehadiran dan status peminjaman terenkapsulasi dengan benar.



Gambar 3.10: Class Diagram Sistem

Berdasarkan rancangan pada Gambar 3.10, sistem terdiri dari beberapa kelas entitas utama dan kelas enumerasi (*Enum*) untuk menjaga konsistensi data. Tabel 3.4 merinci peran setiap kelas sebagai acuan implementasi sistem.

Tabel 3.4: Rincian Kelas dalam Sistem

No.	Nama Kelas	Deskripsi
1	User	Merepresentasikan pengguna sistem dengan peran Mahasiswa, Sarpras, Security, atau Admin
2	Organisasi	Menyimpan data organisasi atau unit yang menaungi pengguna dan kegiatan
3	Ruangan	Menyimpan data ruangan yang dapat digunakan dalam proses peminjaman
4	Barang	Menyimpan data barang atau inventaris yang dapat dipinjam
5	Kegiatan	Menyimpan data kegiatan yang menjadi dasar pengajuan peminjaman

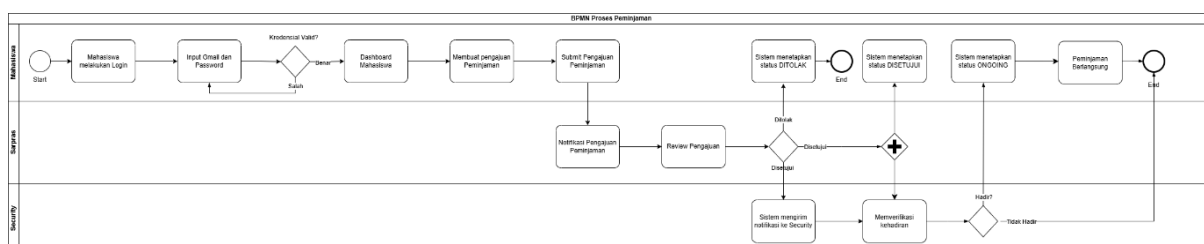
No.	Nama Kelas	Deskripsi
6	Peminjaman	Menyimpan data utama transaksi peminjaman sarana dan prasarana
7	PeminjamanBarangDetail	Menyimpan detail barang yang dipinjam dalam satu transaksi peminjaman
8	KehadiranPeminjam	Menyimpan data verifikasi kehadiran peminjam
9	Mailbox	Menyimpan pesan atau notifikasi sistem yang dikirim kepada pengguna
10	LogAktivitas	Menyimpan catatan aktivitas dan jejak audit pengguna dalam sistem
11	AuthService	Menyediakan layanan autentikasi pengguna
12	PeminjamanService	Menyediakan layanan pengelolaan logika bisnis peminjaman
13	UserRepository	Menyediakan akses data pengguna ke basis data
14	PeminjamanRepository	Menyediakan akses data peminjaman ke basis data

3.5.6 Business Process Model and Notation (BPMN) Diagram

BPMN (Business Process Model and Notation) digunakan untuk memetakan alur proses bisnis secara *end-to-end*, yang menggambarkan interaksi antara berbagai partisipan dalam sistem. Diagram ini memberikan pandangan menyeluruh mengenai bagaimana informasi mengalir dan keputusan diambil antar aktor.

1. BPMN Proses Peminjaman

BPMN proses peminjaman menggambarkan alur kerja utama dalam sistem, dimulai dari inisiasi pengajuan oleh mahasiswa hingga penyelesaian status peminjaman. Proses ini melibatkan tiga *lane* partisipan utama: Mahasiswa, Sarpras (Sarana Prasarana), dan *Security*.



Gambar 3.11: BPMN Diagram Proses Peminjaman

Berdasarkan rancangan pada Gambar 3.11, alur proses berjalan sebagai berikut:

- Inisiasi (Lane Mahasiswa):** Proses dimulai saat mahasiswa melakukan *Login* ke dalam sistem. Sistem memvalidasi kredensial; jika valid, mahasiswa

diarahkan ke *Dashboard*. Mahasiswa kemudian melakukan aktivitas Membuat Pengajuan Peminjaman dengan mengisi formulir digital dan menekan tombol *Submit*.

2. **Verifikasi (Lane Sarpras):** Setelah pengajuan dikirim, sistem secara otomatis mengirimkan Notifikasi Pengajuan ke pihak Sarpras. Staf Sarpras kemudian melakukan aktivitas *Review* Pengajuan untuk memeriksa ketersediaan aset dan kelayakan kegiatan.

- Terdapat *gateway* eksklusif (keputusan):
 - **Jika Ditolak:** Sistem menetapkan status DITOLAK, dan proses berakhir (*End Event*).
 - **Jika Disetujui:** Sistem menetapkan status DISETUJUI, dan alur berlanjut ke validasi lapangan.

3. **Validasi Lapangan (Lane Security):** Setelah disetujui, sistem mengirimkan notifikasi ke Security. Saat kegiatan berlangsung, Security melakukan aktivitas Memverifikasi Kehadiran fisik peminjam.

- Terdapat *gateway* pengecekan kehadiran:
 - **Jika Hadir:** Sistem menetapkan status *ONGOING* (sedang berlangsung) dan peminjaman dianggap aktif hingga selesai.
 - **Jika Tidak Hadir:** Sistem akan menutup proses atau membatalkan akses.

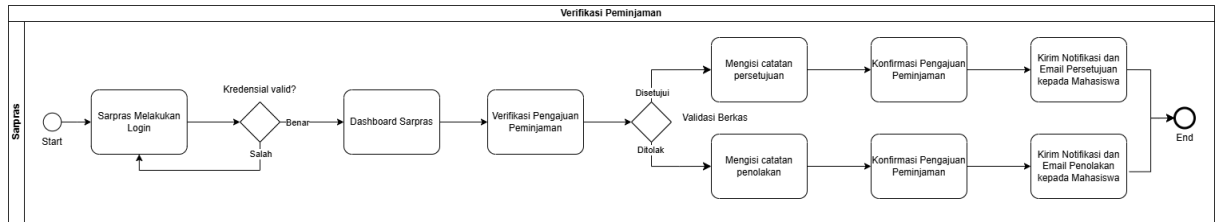
Diagram ini menegaskan bahwa sistem mengintegrasikan tiga departemen berbeda dalam satu alur kerja sekuensial yang otomatis, mengurangi kebutuhan koordinasi manual antar unit.

2. BPMN Verifikasi Peminjaman

BPMN Verifikasi Peminjaman menggambarkan alur kerja administratif yang dilakukan oleh aktor Sarpras (Sarana dan Prasarana) dalam merespons pengajuan yang masuk. Proses ini merupakan titik kontrol utama untuk memastikan ketersediaan aset dan kelayakan kegiatan sebelum izin diterbitkan.

Berdasarkan rancangan pada Gambar 3.12, alur proses berjalan sebagai berikut:

1. **Akses dan Validasi Awal:** Proses dimulai di *lane* Sarpras dengan aktivitas Melakukan *Login*. Sistem memeriksa validitas kredensial; jika valid, Sarpras diarahkan ke *Dashboard* Sarpras untuk melihat daftar pengajuan masuk.



Gambar 3.12: BPMN Diagram Verifikasi Peminjaman

2. **Proses Verifikasi:** Sarpras melakukan aktivitas Verifikasi Pengajuan Peminjaman dengan meninjau detail kegiatan, waktu, dan dokumen pendukung. Pada tahap ini, terdapat *Exclusive Gateway* (keputusan) yang membagi alur menjadi dua cabang berdasarkan hasil tinjauan:

- **Cabang Disetujui:** Jika pengajuan layak, Sarpras memilih opsi “Disetujui”. Sistem kemudian mengarahkan Sarpras untuk Mengisi Catatan Persetujuan (opsional) dan melakukan Konfirmasi Pengajuan. Secara otomatis, sistem akan Mengirim Notifikasi dan Email Persetujuan kepada mahasiswa sebagai bukti izin sah.
- **Cabang Ditolak:** Jika pengajuan tidak layak (misalnya karena bentrok jadwal atau dokumen tidak lengkap), Sarpras memilih opsi “Ditolak”. Sarpras diwajibkan Mengisi Catatan Penolakan sebagai umpan balik. Setelah Konfirmasi Pengajuan, sistem akan Mengirim Notifikasi dan Email Penolakan kepada mahasiswa agar dapat segera direvisi atau diajukan ulang.

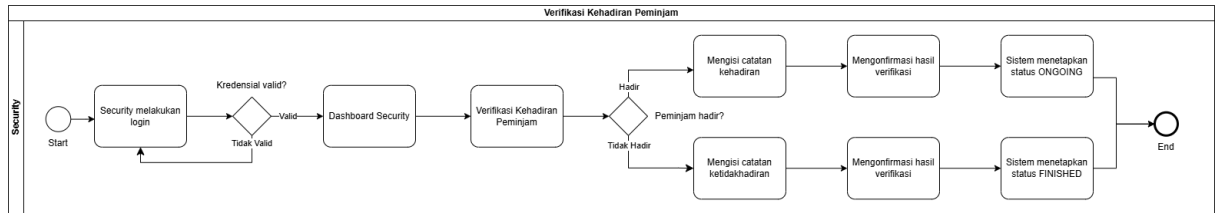
3. **Penyelesaian:** Proses berakhir (*End Event*) setelah notifikasi terkirim, dan status peminjaman di basis data diperbarui menjadi *APPROVED* atau *REJECTED*.

Diagram ini menunjukkan bahwa sistem memfasilitasi transparansi keputusan, di mana alasan penolakan atau detail persetujuan tersampaikan secara *real-time* kepada peminjam, menggantikan proses konfirmasi manual yang lambat.

3. BPMN Verifikasi Kehadiran

BPMN Verifikasi Kehadiran menggambarkan prosedur operasional di lapangan yang dijalankan oleh aktor *Security* (Petugas Keamanan). Proses ini dirancang untuk menjembatani kesenjangan antara data penjadwalan di sistem dengan penggunaan aktual fasilitas fisik (*real-world execution*), memastikan bahwa status penggunaan ruangan di basis data mencerminkan kondisi sebenarnya.

Berdasarkan rancangan pada Gambar 3.13, alur proses berjalan sebagai berikut:



Gambar 3.13: BPMN Diagram Verifikasi Kehadiran

1. **Akses Sistem:** Proses dimulai di *lane* Security dengan aktivitas Security Melakukan *Login*. Sistem memvalidasi kredensial; jika valid, petugas diarahkan ke *Dashboard* Security dan memilih menu Verifikasi Kehadiran Peminjam.
2. **Validasi Lapangan:** Saat jadwal peminjaman tiba, Security melakukan pengecekan fisik apakah peminjam hadir di lokasi. Pada tahap ini terdapat *Exclusive Gateway* (keputusan) “Peminjam Hadir?” yang membagi alur menjadi dua skenario:
 - **Skenario Hadir:** Jika peminjam datang, Security melakukan aktivitas Mengisi Catatan Kehadiran dan Mengonfirmasi Hasil Verifikasi. Sistem kemudian secara otomatis Menetapkan Status *ONGOING*, yang menandakan bahwa fasilitas sedang digunakan secara sah.
 - **Skenario Tidak Hadir:** Jika peminjam tidak datang dalam batas waktu toleransi, Security melakukan aktivitas Mengisi Catatan Ketidakhadiran (sebagai bukti audit) dan Mengonfirmasi Hasil Verifikasi. Sistem kemudian Menetapkan Status *FINISHED* (atau dibatalkan), sehingga ruangan tercatat tidak digunakan dan transaksi ditutup.
3. **Penyelesaian:** Proses berakhir (*End Event*) setelah status sistem diperbarui. Mekanisme ini mencegah klaim penggunaan fiktif dan memastikan aset kampus aman terkendali.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan Implementasi Sistem

4.1.1 Perangkat Keras

Perangkat keras yang digunakan dalam implementasi sistem memiliki spesifikasi sebagai berikut:

1. Laptop: Lenovo LOQ 15 IRX9
2. Prosesor: Intel Core i5-13450HX
3. Kartu Grafis: NVIDIA GeForce RTX 3050 6GB GDDR6
4. Memori: 20 GB DDR5 4800 MHz
5. Penyimpanan Utama: NVMe SSD 512 GB PCIe Gen 4.0 x4

Spesifikasi perangkat keras tersebut dinilai telah memadai untuk menjalankan aplikasi *backend*, basis data, serta proses pengembangan dan pengujian sistem secara bersamaan tanpa mengalami kendala performa yang signifikan.

4.1.2 Perangkat Lunak

Perangkat lunak yang digunakan dalam implementasi sistem meliputi sistem operasi, bahasa pemrograman, basis data, serta berbagai *tools* pendukung pengembangan sebagai berikut:

1. **Sistem Operasi**
 - (a) *Windows 11* sebagai sistem operasi utama pengembangan
2. **Backend**
 - (a) Bahasa pemrograman *Go* versi 1.25.3
 - (b) *PostgreSQL* sebagai sistem manajemen basis data relasional
 - (c) *Supabase* sebagai platform *hosting database* dan penyimpanan *file*
 - (d) *Air* sebagai *development tool* untuk *hot reload* selama pengembangan

3. *Frontend*

- (a) *HTML5* untuk struktur halaman *web*
- (b) *CSS3* untuk pengaturan tampilan dan tata letak
- (c) *JavaScript (Vanilla)* untuk logika interaksi pengguna

4. *Library dan Dependency Backend*

- (a) `github.com/golang-jwt/jwt/v5` untuk implementasi autentikasi berbasis *JSON Web Token*
- (b) `github.com/jackc/pgx/v5` sebagai *driver PostgreSQL*
- (c) `golang.org/x/crypto/bcrypt` untuk enkripsi kata sandi
- (d) `github.com/joho/godotenv` untuk manajemen *environment variables*
- (e) `github.com/xuri/excelize/v2` untuk fitur ekspor laporan ke format *Excel*

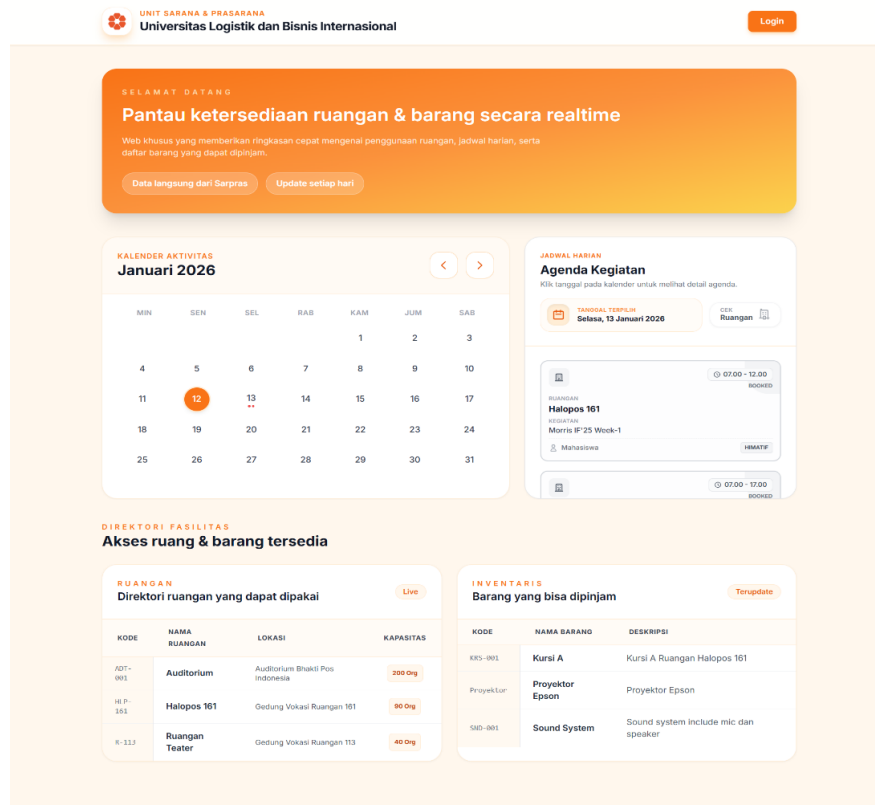
Pemilihan perangkat lunak tersebut didasarkan pada pertimbangan performa, keamanan, kemudahan pengembangan, serta dukungan komunitas yang baik, sehingga mendukung implementasi sistem secara efektif dan berkelanjutan.

4.2 Implementasi Desain Tampilan

4.2.1 Halaman Utama (*Guest*)

Halaman Utama (*Guest*) merupakan halaman publik yang dapat diakses oleh pengguna tanpa melakukan *login* ke dalam sistem. Halaman ini berfungsi sebagai pusat informasi awal yang menampilkan ketersediaan sarana dan prasarana kampus secara umum, sehingga pengguna dapat memperoleh gambaran kondisi fasilitas sebelum mengajukan peminjaman.

Pada Gambar 4.1, ditampilkan informasi berupa kalender aktivitas, jadwal peminjaman harian, daftar ruangan, serta inventaris barang yang tersedia. Seluruh informasi disajikan secara ringkas dan mudah dipahami untuk mendukung transparansi data peminjaman. Dengan adanya Halaman Utama ini, pengguna dapat memantau jadwal penggunaan fasilitas kampus secara efektif dan memperoleh informasi awal tanpa harus masuk ke dalam sistem.

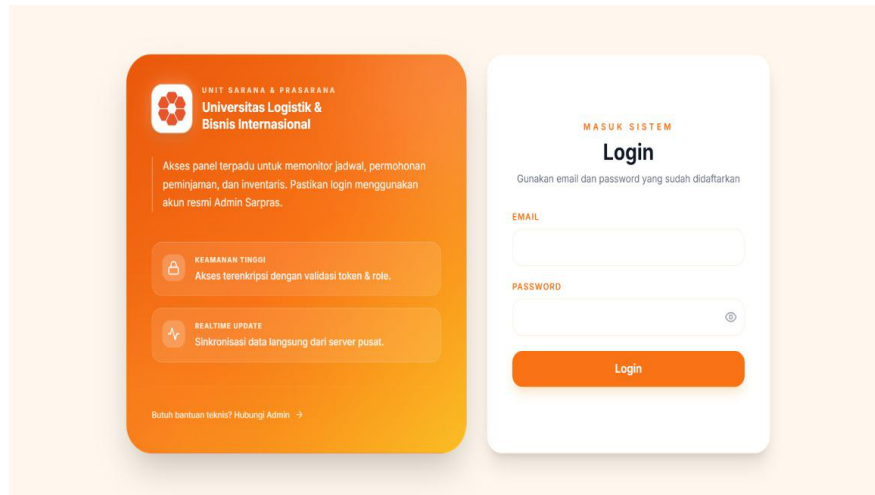


Gambar 4.1: Tampilan Halaman Utama

4.2.2 Halaman Login

Halaman *Login* digunakan sebagai gerbang autentikasi bagi pengguna untuk dapat mengakses sistem sesuai dengan hak akses yang dimiliki. Pada Gambar 4.2, pengguna diminta untuk memasukkan *email* dan kata sandi yang telah terdaftar sebagai proses verifikasi identitas sebelum masuk ke dalam sistem.

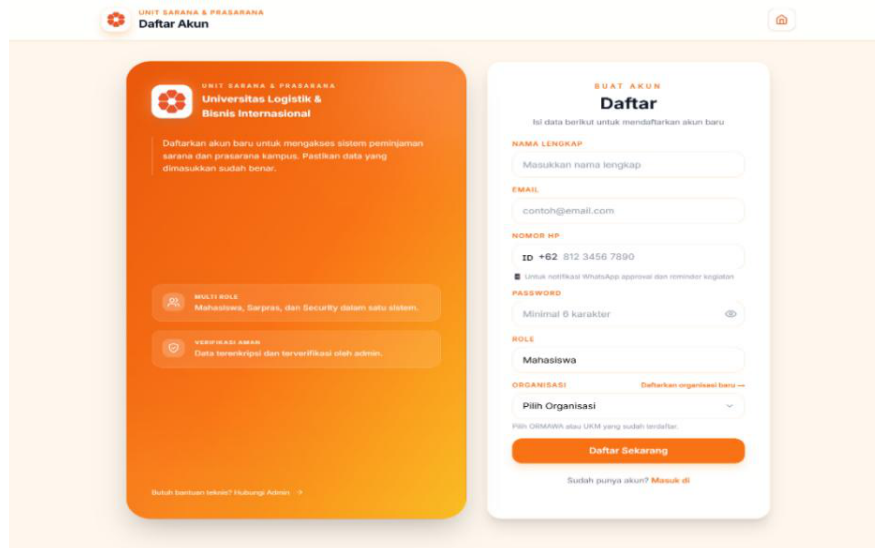
Sistem akan melakukan validasi terhadap data yang dimasukkan oleh pengguna. Apabila data yang diberikan valid, pengguna akan diarahkan ke *dashboard* sesuai dengan peran masing-masing, seperti mahasiswa, admin Sarpras, atau petugas *security*. Sebaliknya, jika data tidak valid, sistem akan menampilkan pesan kesalahan sebagai informasi kepada pengguna. Halaman *Login* dirancang dengan tampilan sederhana dan responsif guna memberikan kemudahan serta kenyamanan dalam proses autentikasi pengguna.



Gambar 4.2: Tampilan Halaman *Login*

4.2.3 Halaman *Register*

Halaman *Register* digunakan oleh pengguna baru untuk melakukan pendaftaran akun sebelum dapat mengakses sistem. Pada Gambar 4.3, pengguna diminta untuk mengisi data diri secara lengkap, seperti nama, *email*, kata sandi, identitas mahasiswa, serta informasi organisasi yang terkait dengan pengajuan peminjaman.

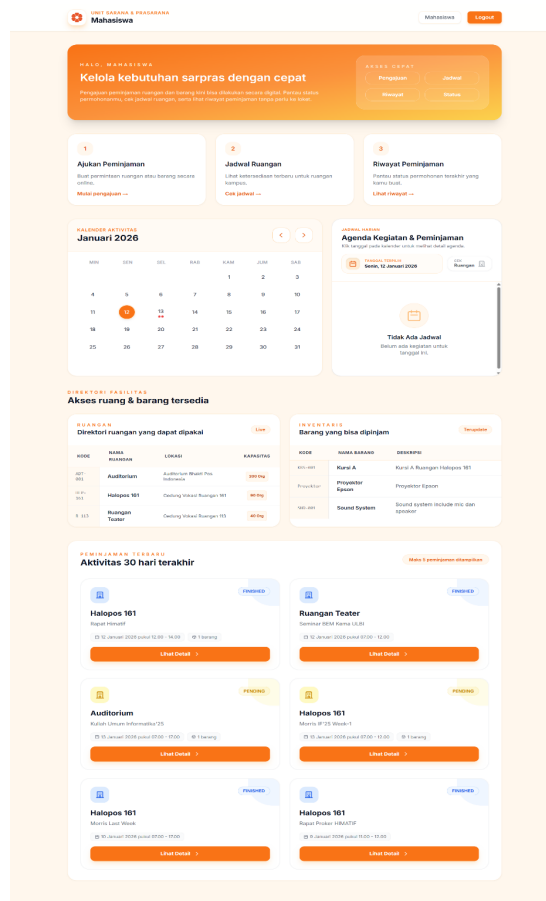


Gambar 4.3: Tampilan Halaman *Register*

Sistem melakukan validasi terhadap data yang dimasukkan untuk memastikan keakuratan dan kelengkapan informasi. Setelah proses pendaftaran berhasil, data pengguna akan disimpan ke dalam sistem dan akun dapat digunakan untuk melakukan *login*. Halaman *Register* dirancang agar mudah dipahami dan digunakan, sehingga proses pembuatan akun dapat dilakukan secara efektif dan efisien oleh pengguna.

4.2.4 Dashboard Mahasiswa

Dashboard Mahasiswa merupakan halaman utama yang diakses oleh mahasiswa setelah berhasil melakukan *login* ke dalam sistem. Halaman ini berfungsi sebagai pusat informasi dan aktivitas mahasiswa dalam proses peminjaman sarana dan prasarana kampus.



Gambar 4.4: Tampilan *Dashboard* Mahasiswa

Pada Gambar 4.4, ditampilkan ringkasan informasi peminjaman, kalender jadwal penggunaan fasilitas, serta agenda harian yang berkaitan dengan peminjaman yang diajukan. Selain itu, mahasiswa dapat mengakses fitur utama seperti pengajuan peminjaman, melihat status dan riwayat peminjaman, serta memantau jadwal penggunaan sarana dan prasarana. *Dashboard* Mahasiswa dirancang untuk memudahkan pengguna dalam memantau dan mengelola aktivitas peminjaman secara terstruktur dan efisien.

4.2.5 Form Pengajuan Peminjaman

Form Pengajuan Peminjaman digunakan oleh mahasiswa untuk mengajukan peminjaman sarana dan prasarana kampus melalui sistem. Pada Gambar 4.5, mahasiswa mengisi data pengajuan yang meliputi informasi kegiatan, pilihan ruangan dan/atau barang yang akan dipinjam, jadwal penggunaan, serta dokumen pendukung peminjaman.

The screenshot shows a web form titled "UNIT SARANA & PRASARANA Form Pengajuan". The main heading is "Jadwal Fleksibel" with a subtext: "Pilih tanggal-tanggal yang kamu butuhkan, lalu atur jam pemakaian yang berbeda untuk setiap tanggalnya." Below this, there are several input fields and sections:

- RUANGAN (OPSIONAL)**: A dropdown menu labeled "Pilih Ruangan (Opsional)". To its right, a note says: "Ruangan kosong? Kamu tetap dapat meminjam barang tanpa memilih ruangan."
- LANGKAH 1: PILIH TANGGAL-TANGGAL ***: A date selection field with the placeholder "Klik untuk memilih tanggal...". Below it, a legend indicates: "Maksimal 2 minggu ke depan", "Hari Minggu tidak tersedia", and "Susah dibooking".
- NAMA KEGIATAN ***: A text input field with the example "Contoh: Rapat Kerja BEM".
- DESKRIPSI KEGIATAN (OPSIONAL)**: A larger text area with the placeholder "Jelaskan secara singkat kegiatan yang akan dilakukan...".
- UNGGAH SURAT DIGITAL (PDF) ***: A file upload section with a "Pilih File" button and the text "Tidak ada file yang dipilih". Below it, a small note states: "Format PDF, maksimal 2MB. Satu surat dapat digunakan untuk semua tanggal yang diajukan."
- BARANG YANG DIPINJAM (OPSIONAL)**: A section with a "+ Tambah Barang" button.

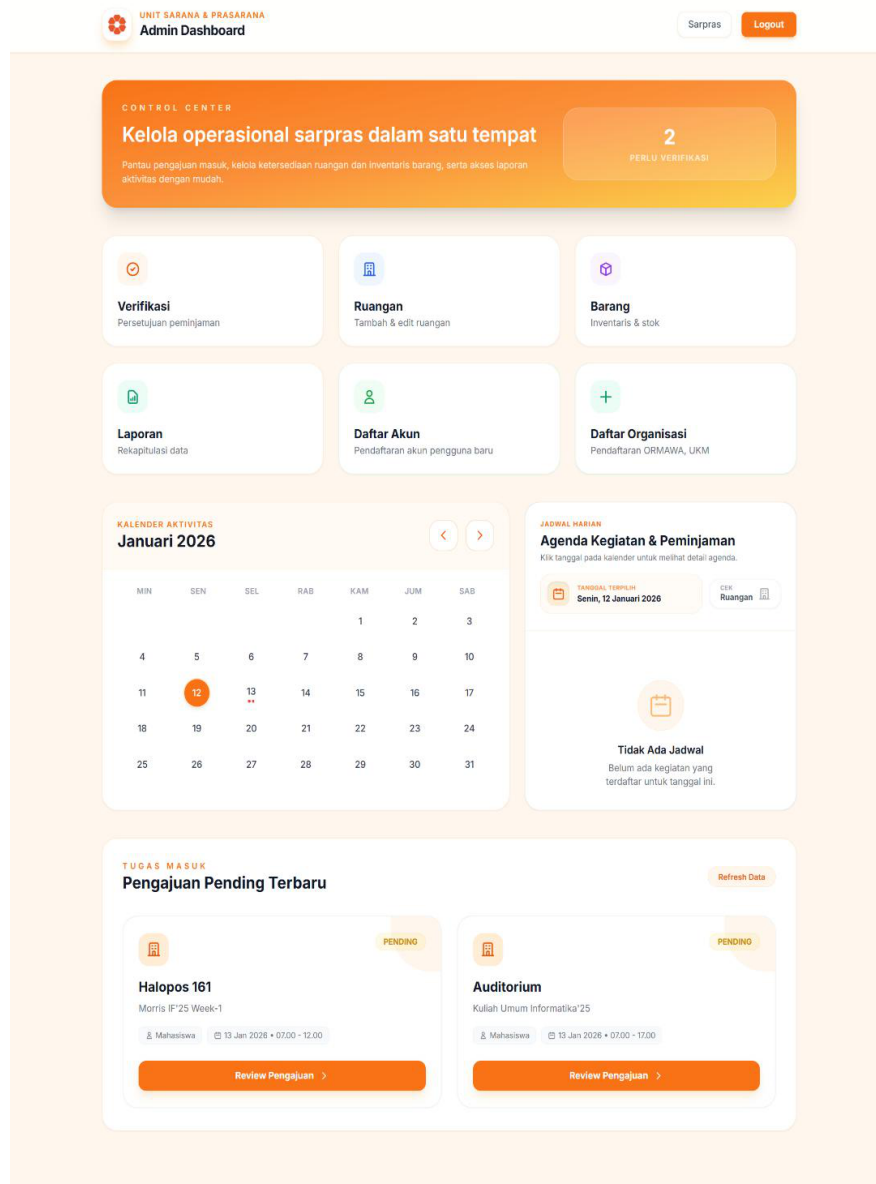
At the bottom of the form are two buttons: "Ajukan Peminjaman" (orange) and "Batal & Kembali" (white with orange border).

Gambar 4.5: Tampilan *Form* Pengajuan

Sistem akan melakukan validasi terhadap data yang dimasukkan sebelum pengajuan diproses. Setelah pengajuan dikirim, data akan disimpan dalam sistem dan menunggu proses verifikasi oleh pihak Sarpras. *Form* ini dirancang secara terstruktur dan mudah digunakan untuk membantu mahasiswa dalam menyampaikan pengajuan peminjaman secara jelas dan terdokumentasi dengan baik.

4.2.6 Dashboard Sarpras

Dashboard Sarpras merupakan halaman utama yang digunakan oleh pihak Sarana dan Prasarana untuk mengelola dan memantau seluruh proses peminjaman dalam sistem. Halaman ini menampilkan ringkasan informasi pengajuan peminjaman, khususnya pengajuan yang masih menunggu proses verifikasi.



Gambar 4.6: Tampilan *Dashboard* Sarpras

Melalui *Dashboard* Sarpras pada Gambar 4.6, admin dapat mengakses fitur manajemen data sarana dan prasarana, melakukan verifikasi pengajuan peminjaman, serta memantau jadwal penggunaan fasilitas kampus. *Dashboard* ini dirancang untuk membantu pihak Sarpras dalam menjalankan tugas pengelolaan peminjaman secara terpusat, efisien, dan terkontrol.

4.2.7 Form Verifikasi Peminjaman

Form Verifikasi Peminjaman digunakan oleh pihak Sarpras untuk melakukan pemeriksaan dan pengambilan keputusan terhadap pengajuan peminjaman yang diajukan oleh mahasiswa. Pada Gambar 4.7, Sarpras dapat melihat daftar pengajuan yang menunggu verifikasi beserta detail informasi peminjaman yang diajukan.

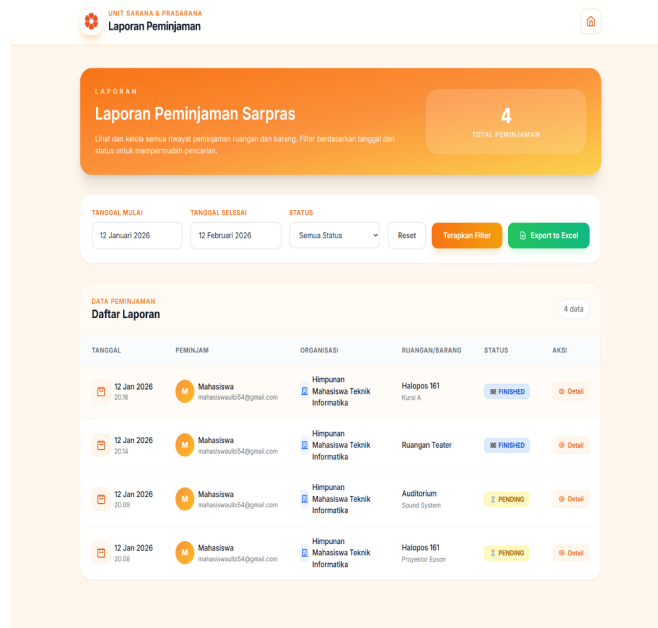
The screenshot displays the 'Form Verifikasi Peminjaman' interface. At the top, it shows the unit name 'UNIT SARANA & PRASARANA' and the title 'Verifikasi Peminjaman'. The main section is titled 'ADMIN PANEL Detail Pengajuan' and shows a pending request for 'Halopos 161'. The request details include the applicant's information (Mahasiswa), the loan schedule (13/1/2026, 14.00.00 — 13/1/2026, 19.00.00), the activity name (Morris IF'25 Week-1), and the item being loaned (Proyektor Epson, 1 unit). There is also a section for supporting documents (Dokumen Pendukung) with a link to the 'Surat Pengajuan'. On the right side, there is a 'Keputusan Verifikasi' section with buttons for 'Setujui' (Approve) and 'Tolak' (Reject), and a 'CATATAN ADMIN' section for adding remarks. A 'Simpan Keputusan' (Save Decision) button is located at the bottom right of the decision section.

Gambar 4.7: Tampilan *Form* Verifikasi Peminjaman

Berdasarkan hasil pemeriksaan data dan dokumen pendukung, Sarpras dapat memberikan keputusan berupa persetujuan atau penolakan pengajuan peminjaman. Setiap keputusan yang diambil akan dicatat oleh sistem dan disampaikan kepada mahasiswa melalui notifikasi sebagai informasi hasil verifikasi. *Form* ini dirancang untuk mendukung proses verifikasi peminjaman secara sistematis, transparan, dan terdokumentasi.

4.2.8 Halaman Laporan Peminjaman

Halaman Laporan Peminjaman digunakan oleh pihak Sarpras untuk melihat rekapitulasi data peminjaman sarana dan prasarana yang tersimpan dalam sistem. Halaman ini menyajikan informasi peminjaman secara terstruktur sehingga memudahkan proses pemantauan dan evaluasi.



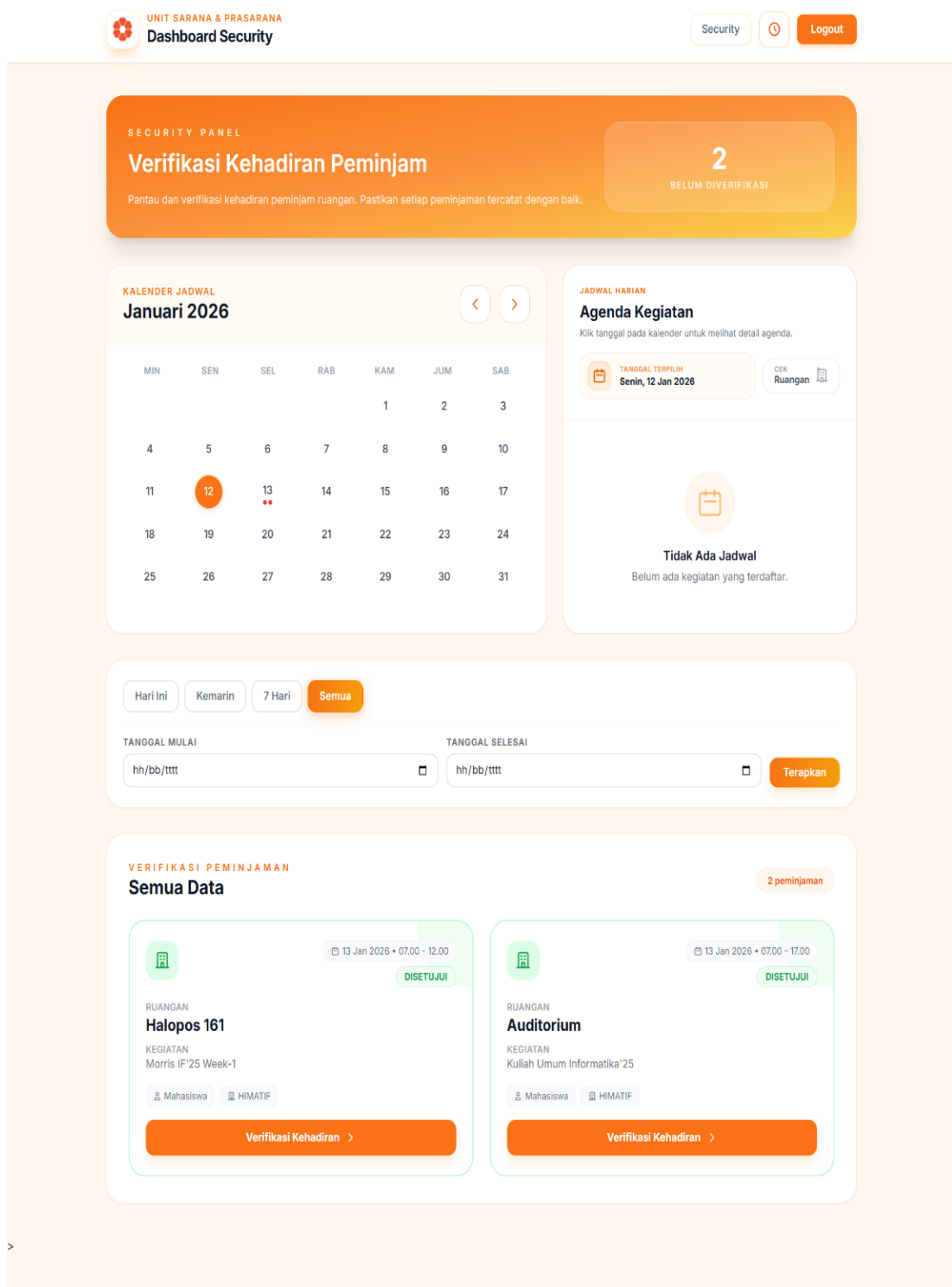
Gambar 4.8: Tampilan Halaman Laporan Peminjaman

Pada Gambar 4.8, pengguna dapat melakukan penyaringan data berdasarkan periode waktu dan status peminjaman. Data laporan dapat digunakan sebagai bahan dokumentasi dan pendukung dalam penyusunan laporan administrasi. Halaman Laporan Peminjaman dirancang untuk membantu pihak Sarpras dalam memperoleh informasi peminjaman secara akurat dan sistematis.

4.2.9 *Dashboard Security*

Dashboard Security merupakan halaman utama yang digunakan oleh petugas keamanan untuk memantau dan memverifikasi kehadiran peminjam sarana dan prasarana kampus. Halaman ini menampilkan daftar jadwal peminjaman yang berlangsung pada hari berjalan beserta status verifikasi kehadiran peminjam.

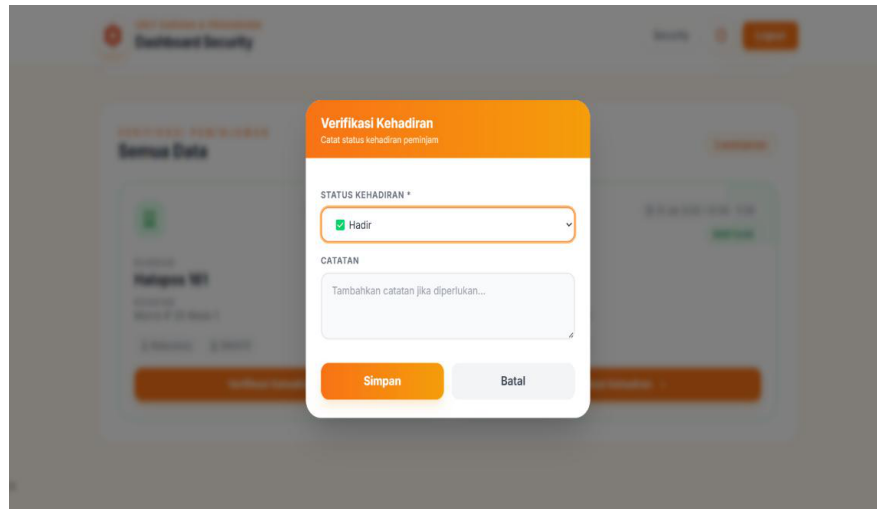
Melalui *Dashboard Security* pada Gambar 4.9, petugas dapat melakukan pengecekan kehadiran peminjam berdasarkan jadwal yang telah disetujui serta mencatat hasil verifikasi ke dalam sistem. *Dashboard* ini dirancang untuk mendukung proses pengawasan penggunaan fasilitas kampus agar sesuai dengan jadwal dan ketentuan yang telah ditetapkan.



Gambar 4.9: Tampilan Dashboard Security

4.2.10 Form Verifikasi Kehadiran

Form Verifikasi Kehadiran digunakan oleh petugas *security* untuk mencatat dan memverifikasi kehadiran peminjam pada saat penggunaan sarana dan prasarana kampus. Pada Gambar 4.10, petugas dapat melihat detail jadwal peminjaman yang sedang berlangsung serta melakukan konfirmasi kehadiran peminjam sesuai dengan data yang tercatat dalam sistem.



Gambar 4.10: Tampilan *Form* Verifikasi Kehadiran

Hasil verifikasi kehadiran akan disimpan dan terdokumentasi sebagai bagian dari riwayat peminjaman. *Form* ini dirancang untuk memastikan penggunaan sarana dan prasarana sesuai dengan jadwal yang telah disetujui serta mendukung pengawasan dan pengendalian fasilitas kampus secara tertib dan terstruktur.

4.3 Pengujian

4.3.1 Pengujian *Black Box*

Pengujian *Black Box* dilakukan dengan memvalidasi *input* dan *output* sistem tanpa melihat struktur internal kode. Pengujian mencakup modul autentikasi, *master data*, peminjaman, kehadiran, notifikasi, *RBAC*, serta pengujian antarmuka pengguna (*UI*). Detail skenario dan hasil pengujian disajikan pada Tabel 4.1. Berdasarkan hasil pengujian tersebut, seluruh skenario uji berhasil dijalankan dengan status *PASS*.

Tabel 4.1: Pengujian Aplikasi

No.	Modul/Fitur	Skenario Uji	Hasil Yang Diharapkan	Hasil Uji
1.	<i>Login</i>	<i>Input email & password valid</i>	Masuk <i>dashboard</i> sesuai <i>role</i>	<i>Passed</i>
2.	<i>Login</i>	<i>Input password salah</i>	Pesan <i>error</i> muncul	<i>Passed</i>
3.	<i>Register</i>	Data valid	Akun berhasil dibuat	<i>Passed</i>
4.	Pengajuan Peminjaman	Data lengkap	Status <i>PENDING</i>	<i>Passed</i>
5.	Verifikasi Peminjaman	<i>Approve</i> data	Status <i>APPROVED</i>	<i>Passed</i>
6.	Pembatalan Peminjaman	Klik batal	Status <i>CANCELED</i>	<i>Passed</i>
7.	Kehadiran	<i>Input</i> kehadiran	Data tersimpan	<i>Passed</i>
8.	<i>RBAC</i>	Akses tanpa hak	Akses ditolak	<i>Passed</i>
9.	Notifikasi	Aksi peminjaman	Notifikasi muncul	<i>Passed</i>
10.	<i>Logout</i>	Klik <i>logout</i>	Kembali ke halaman <i>dashboard guest</i>	<i>Passed</i>

Berdasarkan tabel di atas, dapat disimpulkan bahwa seluruh fungsi sistem telah berjalan sesuai dengan spesifikasi dan tidak ditemukan kegagalan pada proses pengujian fungsional.

4.3.2 Pengujian *Code Coverage*

Pengujian *code coverage* menggunakan *go test* dilakukan untuk mengukur sejauh mana kode program *backend* Sistem Informasi Peminjaman Sarana dan Prasarana Kampus telah dieksekusi selama proses *unit testing*. Pengujian difokuskan pada *layer service* yang merepresentasikan logika bisnis utama sistem.

Hasil pengujian menunjukkan nilai *code coverage* sebesar 31,9%. Nilai ini dipengaruhi oleh beberapa fungsi yang tidak dilakukan pengujian unit secara langsung, seperti konstruktor *service*, fungsi pengiriman *email*, serta ekspor data ke format *Excel* yang bergantung pada layanan eksternal dan *file system*.

Sementara itu, fungsi utama sistem seperti autentikasi pengguna, pengajuan peminjaman, verifikasi peminjaman, pembatalan peminjaman, dan pencatatan kehadiran telah diuji dengan tingkat *coverage* di atas 75%, sehingga dapat disimpulkan bahwa logika bisnis inti sistem telah berjalan dengan baik dan sesuai kebutuhan fungsional. Rincian persentase *coverage* untuk masing-masing *service* disajikan pada Tabel 4.2.

Tabel 4.2: Ringkasan Hasil *Code Coverage Backend*

No	Service	Fungsi Utama	Coverage (%)
1	<i>Auth Service</i>	<i>Login, Register</i>	75 – 100
2	<i>Code Generator</i>	<i>generateCode</i>	100
3	<i>Kehadiran Service</i>	<i>CreateKehadiran</i>	88.2
4	<i>Peminjaman Service</i>	<i>Create, Verifikasi, Cancel</i>	76.9 – 87.5
5	<i>Export Service</i>	<i>Generate Excel</i>	0
6	<i>Email Service</i>	Kirim Notifikasi	0
Total Coverage			31.9

Hasil visual pengujian *Code Coverage* disajikan dalam bentuk laporan *HTML* dan dilampirkan pada bagian Lampiran *Code Coverage* sebagai bukti pendukung.

4.4 *Repository GitHub Project*

Repository GitHub digunakan sebagai media penyimpanan dan pengelolaan *source code* dalam pengembangan Sistem Informasi Peminjaman Sarana dan Prasarana Kampus. *Repository* ini mendukung pengendalian versi dan dokumentasi selama proses pengembangan. Struktur *repository* dipisahkan antara *backend* dan *frontend* untuk memudahkan pengelolaan kode. *Repository* proyek dapat diakses melalui:

- (a) <https://github.com/ditverse/proyek-2-backend>
- (b) <https://github.com/ditverse/proyek-2-frontend>

Penggunaan *GitHub* membantu memastikan proses pengembangan sistem berjalan secara terstruktur dan terdokumentasi dengan baik.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, penelitian ini menghasilkan beberapa kesimpulan utama sebagai berikut:

1. Implementasi Teknis dan Metodologi

Sistem informasi peminjaman sarana dan prasarana kampus berbasis web telah berhasil dibangun menggunakan bahasa pemrograman Go (Golang) untuk *backend* dan PostgreSQL sebagai basis data, yang dikelola menggunakan metode pengembangan Scrum. Berdasarkan pengujian *Black Box*, seluruh fungsi utama seperti autentikasi, pengajuan, dan verifikasi berjalan sesuai spesifikasi dengan tingkat keberhasilan (*Pass*) 100%, serta mencapai *code coverage* di atas 75% pada modul-modul inti layanan.

2. Validasi Kehadiran Fisik (*Physical Attendance Validation*)

Penelitian ini menawarkan kebaruan (*novelty*) melalui mekanisme validasi kehadiran fisik yang melibatkan petugas keamanan (*Security*) sebagai verifikator akhir. Fitur ini dirancang sebagai solusi teknis untuk mengatasi masalah *capacity waste* (pemborosan kapasitas) pada sistem reservasi. Hal ini merespons isu yang diangkat oleh **Baron**, yang menyatakan bahwa tanpa intervensi terhadap perilaku strategis pengguna—seperti memesan namun tidak hadir—sistem layanan rentan mengalami inefisiensi sumber daya yang signifikan karena slot waktu terbuang sia-sia. Jika literatur terdahulu menyarankan intervensi berupa denda atau kebijakan, sistem ini menawarkan pendekatan intervensi fisik dan digital (*closed-loop validation*) untuk memastikan akuntabilitas penggunaan fasilitas.

3. Penjadwalan Tanpa Tumpang Tindih (*Zero-Overlap Scheduling*)

Validasi jadwal otomatis dalam sistem terbukti mampu mencegah terjadinya *double-booking* (pemesanan ganda) secara sistematis. Fitur ini dirancang sebagai solusi teknis untuk mengeliminasi konflik jadwal yang menjadi kelemahan utama sistem pengelolaan fasilitas konvensional. Hal ini merespons temuan Patel and Professor (2024), yang menyoroti bahwa sistem manual atau

semi-digital sangat rentan terhadap inefisiensi operasional akibat *human error*, konflik jadwal, dan kurangnya transparansi data. Jika sistem tradisional gagal menangani pembaruan waktu nyata (*real-time updates*) yang menyebabkan ketidakpastian ketersediaan ruangan, sistem ini menawarkan mekanisme pencegahan konflik berbasis data yang menolak permintaan tumpang tindih secara instan, memastikan integritas data dan kepastian layanan bagi pengguna.

5.2 Saran

Berdasarkan keterbatasan penelitian saat ini dan potensi pengembangan di masa depan, disarankan beberapa hal berikut untuk meningkatkan dampak dan kualitas penelitian selanjutnya:

1. Validasi Kehadiran Berbasis QR Code

Saat ini, validasi kehadiran dilakukan secara manual oleh petugas keamanan melalui pencarian data pada *dashboard* web. Untuk pengembangan selanjutnya, disarankan untuk mengintegrasikan fitur *QR Code Generator* dinamis sebagai tiket masuk digital yang dapat diakses melalui akun pengguna di portal web. Hal ini memungkinkan proses verifikasi yang lebih cepat melalui pemindaian kode tanpa memerlukan aplikasi *mobile* terpisah, sehingga meminimalisir antrian fisik dan kesalahan manusia (*human error*) di pos keamanan. Sejalan dengan saran Patel and Professor (2024), pendekatan ini akan meningkatkan aksesibilitas pengguna dan efisiensi operasional sistem.

2. Manajemen Inventaris Aset dan Mekanisme Pendisiplinan Pengguna

Disarankan untuk mengembangkan modul Pengelolaan Inventaris yang mencakup pemantauan kondisi aset (*asset condition monitoring*) dan pelacakan riwayat pemeliharaan (*maintenance tracking*) secara berkala, guna memastikan hanya aset dalam kondisi baik yang tersedia untuk dipinjam. Sistem juga perlu dilengkapi mekanisme penalti otomatis bagi pengguna yang melakukan pelanggaran seperti *no-show* atau *strategic double-booking*. **Baron** menekankan bahwa intervensi berupa pembatasan akses diperlukan untuk menjaga keadilan sistem dan mencegah pemborosan kapasitas layanan.

DAFTAR PUSTAKA

- Awoyemi, Robert Akinade and Oluwadolapo Richard Awoyemi (2025). "Smart inventory management systems for Nigerian academic libraries". In: *Library Hi Tech News*. ISSN: 07419058. DOI: 10.1108/LHTN-06-2025-0094.
- Baron, Opher, Olga Bountali, Arseniy Gorbushin, and Binyamin Oz (2024). "Are No-Shows All That Bad? Strategic Double-Booking and When to Intervene". In: *Strategic Double-Booking and When to Intervene (August 01, 2024)*.
- Baxter, R. (2006). *Software engineering is software engineering*, pp. 14–18. ISBN: 2008048802. DOI: 10.1049/ic:20040411.
- Herlambang, Admaja Dwi, Aditya Rachmadi, Azri Putri Rahmatika, Dinar Indah Dwi Utami, and Safira Widya Hapsari (2020). "V-MODEL UNTUK PENGEMBANGAN SISTEM INFORMASI MANAJEMEN RUANG RAPAT". In: 7 (2), pp. 313–322. ISSN: 2528-6579. DOI: 10.25126/jtiik.202071893.
- Kim, Jinho, Yonghee Kim, Soorim Oh, Taejin Kim, and Dogyun Lee (2021). "Estimation of environmental impact of paperless office based on simple model scenarios". In: *International Journal of Sustainable Building Technology and Urban Development* 12 (1), pp. 44–60. ISSN: 20937628. DOI: 10.22712/susb.20210005.
- Kommadi, Bhagvan (2019). *Learn Data Structures and Algorithms with Golang: Level up your Go programming skills to develop faster and more efficient code*. Packt Publishing Ltd.
- Luh, Ni, Gede Pivin Suwirmayanti, I Komang, Agus Ady Aryanto, I G A Ngurah, Wisrama Putra, Ni Kadek Sukerti, Rosalia Hadi, Program Studi, and Sistem Komputer (2020). "Penerapan Helpdesk System dengan Pengujian Blackbox Testing Implementation Of Helpdesk System With Blackbox Testing". In: *Jurnal Ilmiah Intech : Information Technology Journal of UMUS* 2 (02), pp. 55–64. ISSN: 2685-4902.
- Mary, Thomson and Nia Febriyani (July 2025). "Peningkatan Keamanan Sistem Informasi Berbasis Laravel 12 dengan Rate Limiting dan Role-Based Access Control (RBAC)". In: *Jurnal Teknologi Dan Sistem Informasi Bisnis* 7 (3), pp. 473–481. ISSN: 2964-2132. DOI: 10.47233/jteksis.v7i3.1976.
- Murugesan, San and G R Gangadharan (2012). "Green IT: An Overview". In: *Harnessing Green IT: Principles and Practices*, pp. 1–22.

- Nabella, Putri, Rudy Herteno, Setyo Wahyu Saputro, Friska Abadi, and Muhammad Itqan Mazdadi (Feb. 2025). “Pengembangan Sistem Manajemen Sarana Dan Prasarana, IT, Serta Laboratorium Di SMK Telekomunikasi”. In: *Jurnal Teknologi Informasi dan Ilmu Komputer* 12 (1), pp. 1–10. ISSN: 2355-7699. DOI: 10.25126/jtiik.2025128649.
- Nachandiya, Nathan, Yusufu Gambo, Neil’s B. Joel, and Philemon Davwar (Dec. 2018). “Smart Technologies for Smart Campus Information System”. In: *Asian Journal of Research in Computer Science*, pp. 1–7. DOI: 10.9734/ajrcos/2018/v2i228738.
- Nugraha, Try Setya and Rifqi Hardian (2021). “RANCANG BANGUN SISTEM INFORMASI COMPANY PROFILE DENGAN MENGGUNAKAN METODE SCRUM PADA PT. HASNA SATYA NEGARA BERBASIS WEB Design and Build Company Profile Information System Using Scrum Method At PT. Hasna Satya Negara WEB-Based”. In: *Jurnal Ilmiah Intech : Information Technology Journal of UMUS* 03 (02), pp. 171–179. ISSN: 2685-4902.
- Patel, Kenil and Meenakshi Prajapati Associate Professor (2024). *A Comprehensive Booking Management System Using Laravel*. Tech. rep.
- Permana, Putu Adi Guna (2015). “Scrum method implementation in a software development project management”. In: *International Journal of Advanced Computer Science and Applications* 6.9, pp. 198–204.
- Pratama, Fanes and Ahmad Farisi (2025). “Analisis Perbandingan Kinerja Backend API Menggunakan PHP, Golang, dan JavaScript”. In: *Techno.Com* 24.1, pp. 153–165. ISSN: 1412-2693. DOI: 10.62411/tc.v24i1.12080.
- Purwati, Nani, Meidana Wahyu Pratama, and Paulus Tofan Rapiyanta (Dec. 2022). “Sistem Informasi Peminjaman Peralatan Jaringan dan Multimedia Berbasis Website di Biro Sistem Informasi UMY”. In: *Infomatek* 24 (2), pp. 119–124. ISSN: 1411-0865. DOI: 10.23969/infomatek.v24i2.6019.
- Putrianasari, Rahmawati, Eko Kuswardono Budiardjo, Kodrat Mahatma, and Teguh Raharjo (Jan. 2024). “Problems in The Adoption of Agile-Scrum Software Development Process in Small Organization: A Systematic Literature Review”. In: *Sinkron* 9 (1), pp. 495–504. ISSN: 2541-044X. DOI: 10.33395/sinkron.v9i1.13271.
- Rahardian, Rifky Lana, Siti Khodijah, and Cindy Atika Rizki (Nov. 2025). “Evaluation of the Usability of the Academic Information System Using the System Usability Scale (SUS) Method”. In: *Journal of Computer Science Artificial*

- Intelligence and Communications* 2 (2), pp. 62–66. ISSN: 3109-3981. DOI: 10.64803/jocsaic.v2i2.62.
- Ramdany, Kevry and Asniati Bahari (2024). *DIGITAL TRANSFORMATION IN HIGHER EDUCATION THROUGH THE IMPLEMENTATION OF ENTERPRISE RESOURCE PLANNING*. Tech. rep., pp. 310–319.
- Rifky, Muhamad, Yusi Tyroni Mursityo, and Bondan Sapta Prakoso (2022). “PENGEMBANGAN APLIKASI SISTEM INFORMASI PENDATAAN BANGUNAN (SIPBANG) PADA DINAS PEKERJAAN UMUM DAN PENATAAN RUANG (DPUPR) KOTA MALANG MENGGUNAKAN FRAMEWORK SCRUM”. In: 9 (1), pp. 69–78. ISSN: 2528-6579. DOI: 10.25126/jtiik.202293877.
- Schwaber, Ken and Jeff Sutherland (2020). *DER VISUELLE SCRUM GUIDE*. Tech. rep.
- Taley, Divyani Shivkumar and Bageshree Pathak (2020). “Comprehensive study of software testing techniques and strategies: a review”. In: *Int. J. Eng. Res* 9.08, pp. 817–822.
- Velasquez-Jimenez, Linett, Claudia Marrujo-Ingunza, Santiago Rubiños-Jimenez, Juan Grados-Gamarra, and Junior Grados-Espinoza (2025). “Implementation of Software For Efficient Inventory Management at A National Peruvian University”. In: *Journal of Applied Engineering and Technological Science (JAETS)* 7.1, pp. 569–581. DOI: 10.37385/jaets.v7i1.7151.
- Zhou, Yuan, Amith Viswanatha, Ammar Abdul Motaleb, Prabin Lamichhane, Kay Yut Chen, Richard Young, Ayse P. Gurses, and Yan Xiao (Mar. 2023). “A predictive decision analytics approach for primary care operations management: A case study of double-booking strategy design and evaluation”. In: *Computers and Industrial Engineering* 177. ISSN: 03608352. DOI: 10.1016/j.cie.2023.109069.

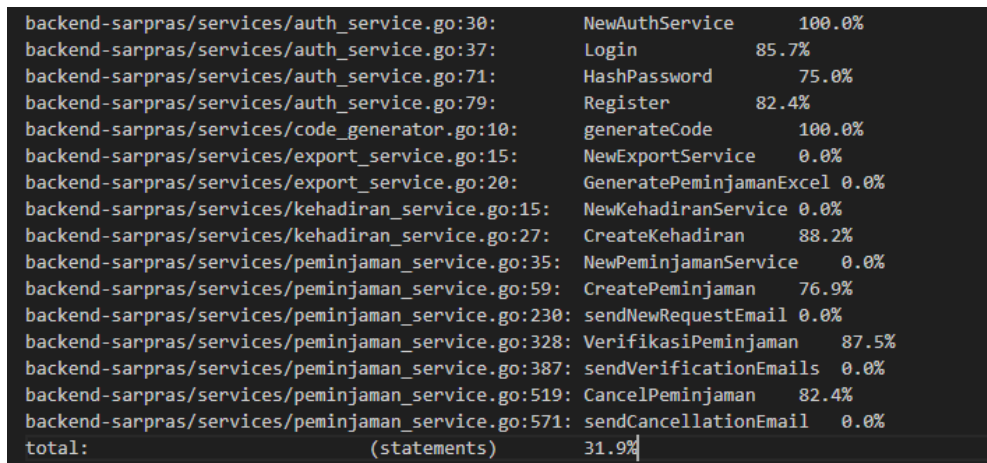
LAMPIRAN

Lampiran A: Hasil Pengujian *Code Coverage Backend*

Lampiran ini menyajikan hasil pengujian *code coverage backend* menggunakan *testing framework* Go. Hasil ditampilkan dalam bentuk laporan HTML yang menunjukkan persentase *coverage* dan detail kode yang telah diuji (*covered*) maupun belum diuji (*not covered*).

A.1 Ringkasan *Code Coverage*

Ringkasan hasil pengujian menunjukkan total *code coverage* sebesar 31,9%. Modul inti (*auth_service*, *code_generator*, *kehadiran_service*, *peminjaman_service*) telah mencapai tingkat keterujian tinggi.



backend-sarpras/services/auth_service.go:30:	NewAuthService	100.0%
backend-sarpras/services/auth_service.go:37:	Login	85.7%
backend-sarpras/services/auth_service.go:71:	HashPassword	75.0%
backend-sarpras/services/auth_service.go:79:	Register	82.4%
backend-sarpras/services/code_generator.go:10:	generateCode	100.0%
backend-sarpras/services/export_service.go:15:	NewExportService	0.0%
backend-sarpras/services/export_service.go:20:	GeneratePeminjamanExcel	0.0%
backend-sarpras/services/kehadiran_service.go:15:	NewKehadiranService	0.0%
backend-sarpras/services/kehadiran_service.go:27:	CreateKehadiran	88.2%
backend-sarpras/services/peminjaman_service.go:35:	NewPeminjamanService	0.0%
backend-sarpras/services/peminjaman_service.go:59:	CreatePeminjaman	76.9%
backend-sarpras/services/peminjaman_service.go:230:	sendNewRequestEmail	0.0%
backend-sarpras/services/peminjaman_service.go:328:	VerifikasiPeminjaman	87.5%
backend-sarpras/services/peminjaman_service.go:387:	sendVerificationEmails	0.0%
backend-sarpras/services/peminjaman_service.go:519:	CancelPeminjaman	82.4%
backend-sarpras/services/peminjaman_service.go:571:	sendCancellationEmail	0.0%
total:	(statements)	31.9%

Gambar L.1: Ringkasan *Code Coverage Backend*

A.2 Detail Coverage: *Auth Service*

Detail *coverage Auth Service* menunjukkan sebagian besar kode telah tercakup pengujian (hijau), termasuk fungsi kritis seperti *login* dan *register*.

```

backend-sarpras/services/auth_service.go (83.3%) not tracked not covered covered
import (
    "backend-sarpras/models"
    "backend-sarpras/repositories"
    "errors"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "golang.org/x/crypto/bcrypt"
)

// UserRepositoryInterface defines the contract for user repository operations
// This allows dependency injection of mock repositories for testing
type UserRepositoryInterface interface {
    Create(user *models.User) error
    GetByEmail(email string) (*models.User, error)
    GetByID(kodeUser string) (*models.User, error)
    GetByRole(role models.RoleEnum) ([]models.User, error)
}

// Compile-time check: ensure *repositories.UserRepository implements UserRepositoryInterface
var _ UserRepositoryInterface = (*repositories.UserRepository)(nil)

type AuthService struct {
    UserRepo UserRepositoryInterface
    JWTSecret string
}

func NewAuthService(userRepo UserRepositoryInterface, jwtSecret string) *AuthService {
    return &AuthService{
        UserRepo: userRepo,
        JWTSecret: jwtSecret,
    }
}

```

Gambar L.2: Detail Coverage Auth Service

A.3 Detail Coverage: Kehadiran Service

Detail coverage *Kehadiran Service* menunjukkan tingkat coverage tinggi (88,2%), dengan fungsi utama CreateKehadiran telah diuji menyeluruh.

```

backend-sarpras/services/kehadiran_service.go (83.3%) not tracked not covered covered
package services

import (
    "backend-sarpras/models"
    "backend-sarpras/repositories"
    "errors"
)

type KehadiranService struct {
    KehadiranRepo KehadiranRepositoryInterface
    PeminjamanRepo PeminjamanRepositoryInterface
    LogRepo LogAktivitasRepositoryInterface
}

func NewKehadiranService(
    kehadiranRepo *repositories.KehadiranRepository,
    peminjamanRepo *repositories.PeminjamanRepository,
    logRepo *repositories.LogAktivitasRepository,
) *KehadiranService {
    return &KehadiranService{
        KehadiranRepo: kehadiranRepo,
        PeminjamanRepo: peminjamanRepo,
        LogRepo: logRepo,
    }
}

func (s *KehadiranService) CreateKehadiran(req *models.CreateKehadiranRequest, securityKode string) error {
    peminjaman, err := s.PeminjamanRepo.GetByID(req.KodePeminjaman)
    if err != nil {
        return err
    }
    if peminjaman == nil {
        return errors.New("peminjaman tidak ditemukan")
    }
}

```

Gambar L.3: Detail Coverage Kehadiran Service

A.4 Detail Coverage: Peminjaman Service

Detail coverage *Peminjaman Service* menunjukkan tingkat coverage baik untuk fungsi inti (pengajuan, verifikasi, pembatalan peminjaman).

```
backend-sarpras/services/peminjaman_service.go (34.7%) not tracked not covered covered
import (
    "errors"
    "fmt"
    "log"
    "time"

    internalsvc "backend-sarpras/internal/services"
    "backend-sarpras/models"
    "backend-sarpras/repositories"
)

// EmailServiceInterface defines email service operations
type EmailServiceInterface interface {
    IsEnabled() bool
    SendEmail(to, subject, htmlBody string) error
}

// Compile-time check for email service
var _ EmailServiceInterface = (*internalsvc.EmailService)(nil)

type PeminjamanService struct {
    PeminjamanRepo PeminjamanRepositoryInterface
    BarangRepo      BarangRepositoryInterface
    NotifikasiRepo  NotifikasiRepositoryInterface
    LogRepo         LogRepositoryInterface
    UserRepo        UserRepositoryInterface
    KegiatanRepo    KegiatanRepositoryInterface
    OrganisasiRepo  OrganisasiRepositoryInterface
    RuanganRepo     RuanganRepositoryInterface
    EmailService    EmailServiceInterface
}

func NewPeminjamanService(
    peminjamanRepo PeminjamanRepositoryInterface,
```

Gambar L.4: Detail Coverage Peminjaman Service

Lampiran B: Dokumentasi Wawancara

Dokumentasi wawancara dengan staf Sarana dan Prasarana sebagai bagian proses pengumpulan data dan analisis kebutuhan sistem.



Gambar L.5: Dokumentasi Wawancara dengan Staf Sarana dan Prasarana

GLOSARIUM

Istilah Teknis

A

API Antarmuka pemrograman aplikasi yang memungkinkan pertukaran data antar sistem perangkat lunak.

B

Backend Bagian sistem yang berjalan di server untuk memproses logika bisnis dan mengelola basis data.

F

Frontend Bagian sistem yang tampil di sisi pengguna untuk menangani antarmuka dan interaksi.

J

JWT JSON Web Token, standar untuk mentransmisikan informasi terenkripsi secara aman antar sistem.

R

Realtime Penyajian atau pemrosesan data secara langsung tanpa jeda waktu signifikan.

Responsive Kemampuan tampilan antarmuka untuk menyesuaikan dengan berbagai ukuran layar.

Istilah Non-Teknis

K

Kampus	Lingkungan atau kompleks bangunan institusi pendidikan tinggi beserta fasilitasnya.
Kehadiran	Keberadaan atau kedatangan peminjam pada waktu dan tempat yang telah dijadwalkan.

M

Mahasiswa	Peserta didik yang terdaftar dan menempuh pendidikan di perguruan tinggi.
-----------	---

P

Peminjaman	Proses menggunakan fasilitas milik pihak lain untuk jangka waktu tertentu.
Prasarana	Fasilitas dasar pendukung yang tidak digunakan langsung dalam kegiatan pendidikan.

S

Sarana	Fasilitas yang digunakan secara langsung untuk menunjang kegiatan pendidikan.
Sarpras	Singkatan dari Sarana dan Prasarana, unit pengelola fasilitas di lingkungan kampus.
Security	Petugas keamanan yang bertugas menjaga ketertiban dan keamanan lingkungan kampus.

U

UKM	Unit Kegiatan Mahasiswa, organisasi yang menampung minat dan bakat mahasiswa.
-----	---

V

Verifikasi	Proses pemeriksaan untuk memastikan kebenaran atau kesesuaian suatu data atau informasi.
------------	--